Learned selection for exact Feynman integral reduction with Kira

Rushil Saraswat

October 14, 2026

Abstract

Reducing Feynman integrals to master integrals is an algebraic step in higher-order perturbative calculations. We investigate whether machine learning can reduce the time required by Kira while retaining exact reduction coefficients. We first study models for selecting seeds, elimination rows, and individual equations. Seed and equation selection reduce time in specific test domains, whereas learned row selection gives no reliable saving and is excluded from the retained method. We then train models to choose seed limits, intermediate master bases, and the number of Kira workers before each reduction begins. On the final test of seven known graph structures at one through three loops, worker selection reduces the sum of batch median times from 82.21 to 78.73 seconds, a saving of 4.2% over the preceding one-worker method. The saving across all repetitions is 2.2%, with an 18.9% increase in CPU use. All 900 timed reductions and 50 additional applications pass exact checks. Separate studies show a 48.3% saving from combining compatible requests and a 543.8-fold speedup from exact recurrence relations in a restricted vacuum family through twelve loops. These results show where learned choices and mathematical reuse improve the tested calculations, while also identifying cases in which prediction or parallel execution adds more cost than it removes.

1 Introduction

Higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes require integrals over loop momenta. As the loop order increases, a calculation produces many integrals with related numerator and denominator structures. Integration-by-parts (IBP) identities connect these integrals and allow them to be expressed in terms of a smaller set of master integrals. This reduction is an essential step in many multiloop calculations in quantum field theory [1, 2].

Kira automates the reduction by generating algebraic identities and solving the resulting linear system [3, 4]. Although the identities are exact, the amount of work depends on which equations are generated and how the system is solved. Modern Kira versions already use improved seed and equation selection to reduce unnecessary work [5]. We call the requested integrals the targets and ask whether models trained on earlier reductions can reduce the time needed to express them in a master basis.

We investigate three places where a learned choice can affect the calculation: selecting the seeds from which IBP identities are generated, selecting the row used in an elimination step, and selecting individual equations before their symbolic coefficients are constructed. Earlier work has used machine-assisted search to discover seed-selection rules [6]. Here we assess each model through the time required to complete the reduction, including prediction, exact checks, and any repeated calculation after an unsuccessful choice. This criterion matters because an accurate prediction or a smaller equation set can still lead to a slower calculation.

The study also examines how these decisions interact with Kira. We train models to choose a complete strategy before equation generation, compare shared reductions for compatible requests, and test exact recurrence relations in a restricted family. The resulting evidence supports different

methods in different domains: some learned choices save time, row prediction gives no reliable gain, and the largest saving in the vacuum study comes from mathematical reuse.

We first define the notation used in this paper and explain the reduction procedure. We then describe the training data and comparison rules before presenting the three local models and the models that select complete strategies. The final sections bring the results together and state their physical scope. The appendices provide the integral definitions, model specifications, derivations, and conditions needed to reproduce each comparison.

2 Feynman integrals and their reduction with Kira

2.1 Families, indices, and sectors

An integral family specifies a set of propagators and the external scalar products. Integrals in the same family differ in the integer powers assigned to those propagators [1, 4]:

$$I_{\boldsymbol{a}}(\boldsymbol{x}, d) = \int \prod_{h=1}^{L} \frac{d^d \ell_h}{i\pi^{d/2}} \prod_{i=1}^{N} D_i^{-a_i}, \qquad D_i = q_i^2 - m_i^2 + i0. \tag{1}$$

Here L is the number of loops, ℓ_h is the momentum of loop h, and N is the number of inverse propagators, including any auxiliary entries. The momentum q_i is a linear combination of loop and external momenta, and m_i is the corresponding mass. The term $i0$ specifies the Feynman prescription. We use dimensional regularization with $d = 4 - 2\epsilon$, where ϵ is the regulator, and collect the independent momentum invariants and squared masses in $\boldsymbol{x}$.

The index a_i determines how D_i enters the integrand. A positive index places it in the denominator, a zero index removes it, and a negative index places a power of it in the numerator. Auxiliary inverse propagators represent scalar products that cannot be expressed using the physical propagators alone. Integrals with the same set of positive indices belong to the same *sector* [4]. These definitions give Kira a common way to label the integrals and organize the equations between them.

2.2 From an IBP identity to a master-integral reduction

An IBP identity starts from an index vector $\boldsymbol{\sigma}$, called a *seed*. We multiply its integrand by a loop or external momentum v, differentiate with respect to a loop momentum ℓ_k, and integrate the total derivative. Dimensional regularization gives [2]

$$0 = \int \prod_{h} \frac{d^d \ell_h}{i\pi^{d/2}} \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_i D_i^{-\sigma_i} \right) = \sum_j A_{\boldsymbol{\sigma},o;j}(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}}. \tag{2}$$

The label o specifies the derivative through the choices of k and v, while j labels the integrals produced when the derivative is expanded. The coefficient $A_{\boldsymbol{\sigma},o;j}$ multiplies integral $I_{\boldsymbol{a}^{(j)}}$ in that identity. Since several operators can act on the same seed, one seed generally generates several equations. Appendix A derives the shifts in the integral indices and the row operation used to eliminate an integral.

Kira generates these equations within prescribed index limits, applies symmetry relations, removes redundant equations, and solves the remaining system [4, 5]. Each equation forms a row of the coefficient matrix, and each integral corresponds to a column. Elimination combines rows to remove integrals in the chosen order; back-substitution then expresses the requested targets in terms of the remaining masters. Kira can manipulate symbolic rational functions directly or use exact arithmetic modulo a prime, with reconstruction when a symbolic result is required [4, 7]. Arithmetic modulo a prime takes place in a finite field, so its numerical appearance does not introduce floating-point error.

A requested target may be a single indexed integral or a linear combination $T = \sum_u c_u I_{\boldsymbol{a}^{(u)}}$. The desired result is

$$T = \sum_{m=1}^{n_M} r_{Tm}(\boldsymbol{x}, d) M_m, \qquad r_{Tm} \in \mathbb{Q}(\boldsymbol{x}, d). \tag{3}$$

Here c_u is the coefficient of a target term, M_m is master integral m, and n_M is the number of masters. The field $\mathbb{Q}(\boldsymbol{x}, d)$ consists of rational functions of the invariants, squared masses, and dimension with rational-number coefficients. Reduction determines the coefficients r_{Tm}; evaluating the master integrals is a separate part of the amplitude calculation.

2.3 Where learned choices can reduce the work

Table 1 identifies the three decisions studied here. Seed selection changes which groups of identities are available, row selection changes the elimination steps, and equation selection can suppress an individual seed–operator pair before Kira constructs its coefficients. We use *native choices* to mean Kira's own decision rules. A learned model changes a decision, while Kira continues to carry out the exact algebra.

Every completed target must reach the prescribed master basis. In the comparisons, we also check its coefficients against an independently obtained exact reduction. If a chosen seed set or equation set is insufficient, the calculation repeats with native choices or a larger seed region. We call this repeated attempt a *fallback* and include its cost in the reported time. Thus the relevant question for each model is whether the work it avoids exceeds the cost of prediction and any fallback.

Table 1: The three decisions and the work each model can avoid.

Model	Decision	Possible saving
Seed selection	Which index vectors supply IBP identities?	Generate fewer groups of equations.
Row selection	Which allowed row eliminates the next integral?	Reduce intermediate expressions and operations.
Equation selection	Which seed–operator pairs are useful?	Avoid constructing unnecessary coefficients.

3 Training data and experimental design

3.1 Constructing targets with known reductions

To train the original models, we need targets together with master coefficients and a recorded sequence of valid reduction steps. We obtain these data by working backwards from checked master relations. If $E_q = \sum_i A_{qi} I_i = 0$ is a source identity, we construct

$$T = \sum_m r_m M_m + \sum_q \lambda_q E_q, \qquad \lambda_q, r_m \in \mathbb{Q}(\boldsymbol{x}, d). \tag{4}$$

Expanding the right-hand side produces an expression in indexed integrals while the added zero identities leave its master coefficients unchanged. We record the identities, their seeds and operators, and the operations used in the construction. Appendix B proves the relation and explains how to recover a reduction path from this record.

This procedure usually produces a sum of integrals. It produces a single indexed integral only when the expanded expression has that form. It also supplies one sufficient seed set and one valid reduction path; it does not establish that either has minimum cost. The labels therefore teach

the original models to reproduce a checked construction, and later timing experiments determine whether their choices help Kira.

3.2 Coverage and separation of training and test data

Table 2 summarizes the original collection. The 17,100 family instances contain 57,979 master entries, counted separately in each family, and 2,301,272 targets. Many instances are parameter variations of the same graph. We call a specified assignment of masses and momentum invariants a *parameter profile*. In particular, the higher-loop additions use the restricted vacuum family derived in Appendix D; they do not represent general scattering graphs through twelve loops. The test set contains 148,494 single integrals and 541,888 integral expressions. Appendices F and G specify the families and the checks used to establish their master bases.

Table 2: Original constructed data. Family instances include parameter variations of the same graph.

Part	Family instances	Targets
Training	11,963	1,610,890
Testing	5,137	690,382
Total	17,100	2,301,272

Related mass assignments remain in the same training or test group at every loop order. The seed and row networks each receive twenty additional passes through the expanded training set, starting from their earlier parameters and optimizer state. Some test profiles had already appeared in earlier evaluations, so these results concern reserved parameter groups rather than entirely new graph structures. The later native-Kira studies use separate training, validation, and test samples: training fits the models, validation selects their settings, and the reserved test measures those fixed choices.

3.3 What the timing comparisons measure

Within each comparison, every method receives the same targets and prescribed master basis and uses the same allocated resources. We use *complete reduction time* for the measured time that includes input preparation, model prediction, algebra, checks, and all fallback attempts. Offline training and the construction of reference answers are excluded. The main comparisons repeat each calculation five times and use the median to reduce the influence of an unusually fast or slow run.

The baseline also needs to be specified. Most native comparisons use the same modified Kira 3.1 executable for learned and native choices, so improvements shared by that executable do not count as a benefit of the model. The cumulative vacuum comparison instead uses the saved original executable as its baseline. Later sections also distinguish the sum of batch median times from a geometric mean that gives each graph equal weight. The latter averages logarithms of time ratios and then exponentiates the result. Appendix E defines these measures and their descriptive uncertainty intervals. Keeping these conditions explicit allows each speedup to be interpreted against the calculation it actually measures.

4 Learning the three reduction decisions

We first test the three decisions separately. For each model, we describe the information it receives, the reference labels used in training, and its effect on exact reduction time. This separates the ability to reproduce a recorded choice from the ability to accelerate Kira.

4.1 Model 1: selecting seeds

The seed network receives the terms of the target, its family, the master indices, and a finite list of candidate seeds. It maps each target term to 32 learned numerical features and combines these with ordered pairs of terms and information about the family and masters. A seed-dependent attention operation then identifies which target terms are most relevant to that candidate. The shared representations and attention follow Refs. [8, 9]; Appendix C gives the full architecture.

The network has 28,097 fitted parameters and assigns a separate score to every candidate:

$$p_b = \frac{1}{1 + e^{-g_\theta(T,F,\mathcal{M},\boldsymbol{\sigma}_b)}}, \qquad \widehat{\mathcal{S}}(T) = \{\boldsymbol{\sigma}_b \in \mathcal{C} : p_b \geq \tfrac{1}{2}\}. \tag{5}$$

Here F denotes the family, $\mathcal{M}$ the master list, $\mathcal{C}$ the candidate list, and g_θ the output of the network with fitted parameters θ. The score estimates whether a seed belongs to the recorded sufficient set. Because a target can require several seeds, several scores can exceed the selection threshold and their sum need not be one. Training uses binary cross-entropy and AdamW [10]. The network receives neither the target's final master coefficients nor its later reduction steps.

On the 690,382 reserved targets, the model has precision 95.750%, recall 96.892%, and F_1 score 96.318%. Its complete predicted set matches the recorded set for 84.332% of targets. Kira reduces 652,517 targets with the predicted sets, corresponding to 94.515% sufficiency, and reduces all targets when the full equation pool is available. These percentages differ because matching the recorded set and supplying enough equations are different tests: another seed set can also be sufficient. Appendix E defines the metrics. We report precision and recall because the large number of negative labels can make overall classification accuracy misleading [11].

To determine whether this prediction saves time, we use a separate comparison on existing identity pools from twelve reserved one-loop box instances. The weights remain fixed, and the timing test uses threshold 0.01, chosen before that comparison; the earlier label and sufficiency tests use 1/2. Both the learned selector and a distance-based rule receive the same basis check and full-pool fallback. For single targets, the learned-to-full-pool time ratio is 0.914, with descriptive 95% interval $[0.871, 0.958]$, giving 8.6% less time. Relative to the distance rule, the ratio is 0.913 with interval $[0.855, 0.972]$, giving 8.7% less time. Every method completes all five repetitions of these twelve cases.

The gain is less clear for larger requests. For sixteen-target batches, the ratio is 0.952 with interval $[0.772, 1.148]$, which includes equal time. Across the full planned seed comparison, 345 trials finish exactly, three remain incomplete, and twelve repetitions are skipped; the timing summary uses only complete paired cases. The identity pools already exist when these timers start. A separate native test that includes IBP generation gives ratio 1.096 with interval $[0.963, 1.275]$. Thus the original network demonstrates a saving for the stated supplied-equation single-target test, while native generation requires a different assessment. Section 4.4 reports new seed trees trained directly on scattering reductions.

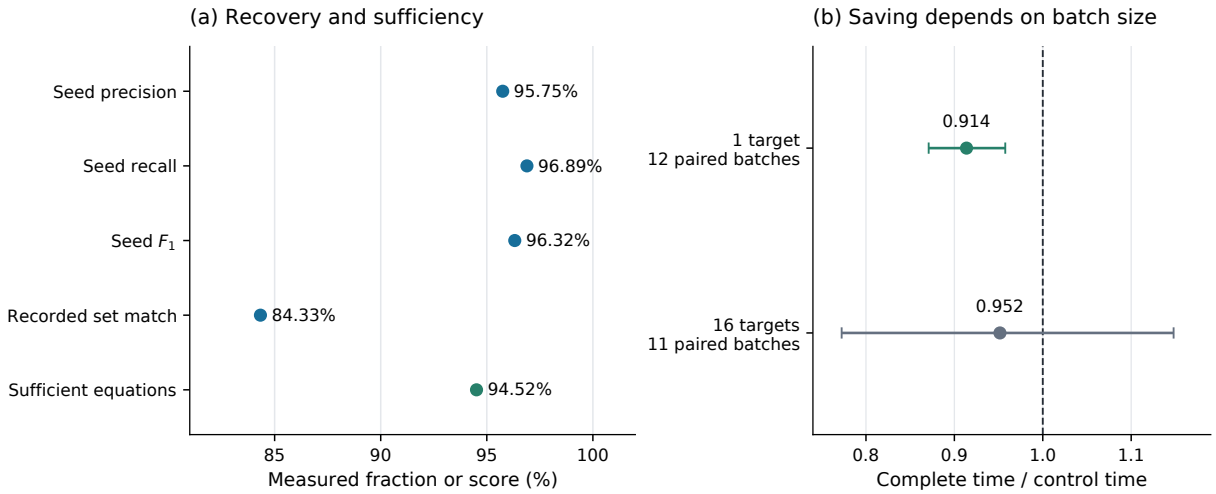


Figure 1: Seed prediction accuracy and reduction speed measure different outcomes. Left: results on the original reserved targets. Right: learned time divided by full-pool time. Values below one are faster. The single-target interval lies below one; the sixteen-target interval crosses it.

4.2 Model 2: selecting elimination rows

After the equations have been generated, several allowed source rows may eliminate the next integral. The row network scores these alternatives using 132 inputs that describe the pivot integral, nonzero entries, family, matrix size, and sampled cancellations. It maps each action to 32 learned features, combines information across the available actions, and passes the result through a hidden layer with 64 components. A softmax converts the final scores into probabilities that sum to one, and the network selects the action with the largest probability. The 10,529 fitted parameters determine the choice; exact arithmetic performs the row operation. Appendices A and C give the operation and architecture.

The original training examples follow one changing target row while the source identities stay fixed. On 2,558,472 reserved states with a choice, the network selects the recorded action in 95.845% of cases, compared with 99.473% for a local deterministic rule. Complete predicted paths agree with the saved master row at three finite-field points for 690,302 of 690,382 targets. These sampled checks support agreement at those points but do not prove symbolic equality. Including prediction, the network takes 17.657 times as long as the local rule.

The more relevant application places the fixed network inside Kira's changing symbolic matrix and preserves Kira's allowed integral order. Across 84 exactly checked trials, this implementation takes 1.305 times the time of stock Kira. We therefore also test smaller policies trained on measured algebraic cost, with compiled prediction and shared features. The latest row-only comparison gives ratio 0.982 with interval $[0.959, 1.003]$, while combining it with the seed policy gives 1.100 with interval $[0.966, 1.279]$ against native choices.

Figure 2 shows that the later intervals still include equal time. We therefore retain Kira's row choices in the final method. This result explains why the remaining studies focus on decisions that can avoid equation generation or repeated work: reproducing a plausible elimination path did not, by itself, provide a reliable reduction in complete time.

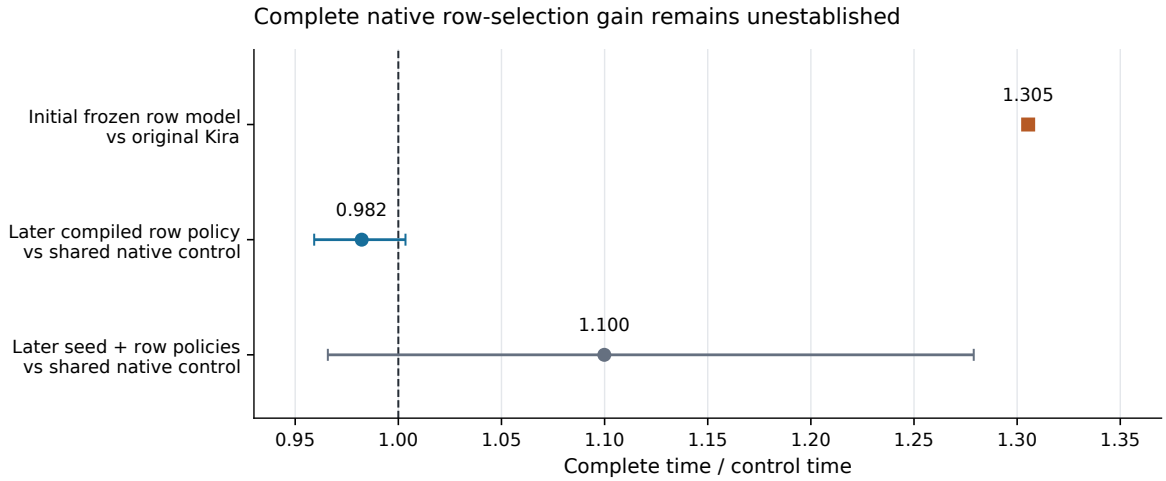


Figure 2: The row model gives no reliable complete-time saving. The first model is slower, and the later row-only interval includes equal time. The original summary provides no interval for the first result. These comparisons use different model versions.

4.3 Model 3: selecting individual equations

Equation selection offers an earlier opportunity to avoid work. The model scores each seed–operator pair before Kira constructs its symbolic coefficients. Its seventeen inputs describe the family, target list, seed indices, and operator; they include the number of loops and targets, extra propagator powers, numerator rank, and distance between seed and target indices. A classification tree [12] applies feature-threshold decisions and returns a score from its final leaf. The score estimates whether the equation belongs to the useful part of a recorded native reduction. No equation coefficients are needed for prediction, and Kira retains its native treatment of symmetries and masters.

The original fit uses 153,124 candidate equations, including 12,270 positive labels, from fourteen batches in eight parameter profiles. The four graph structures are the two-loop sunrise, planar double box, three-loop vacuum, and three-loop ladder. Validation compares trees of depth eight and twelve, containing 295 and 1,335 nodes, at threshold 0.01. It selects the deeper tree for the double box and the shallower tree for the vacuum and ladder; the 21-target sunrise case retains the preceding equation policy. A filter is enabled only when validation shows lower complete and preparation times without fallback. Adding learned seed or row choices fails this condition, so native seed and row choices remain active in this comparison.

Testing uses eight new parameter profiles and fourteen target batches. The masses and momentum invariants take rational values, while d remains symbolic, so the coefficient field is $\mathbb{Q}(d)$. Five repetitions of four methods give 280 complete calculations. Native choices, the early equation model, and the distance-based control share the same executable improvements; a fourth method uses the preceding release. This design distinguishes the effect of equation selection from improvements in the surrounding implementation.

Relative to native choices in that executable, the model reduces complete time by 5.7% and equation-preparation time by 36.2%. The corresponding ratios are 0.943 with interval $[0.915, 0.970]$ and 0.638 with interval $[0.546, 0.734]$. Preparation includes modular generation, selection, modular solving, and symbolic equation regeneration. The smaller change in complete time reflects the work that remains after preparation. All 280 calculations succeed on their first attempt, and independent exact checks cover 94,970 coefficient comparisons and eight applications supplied only with the prescribed basis.

To test how far the original tree transfers, we also apply the fixed depth-twelve model to three structures absent from its training data: a massless nonplanar two-loop box, a massless pentagon,

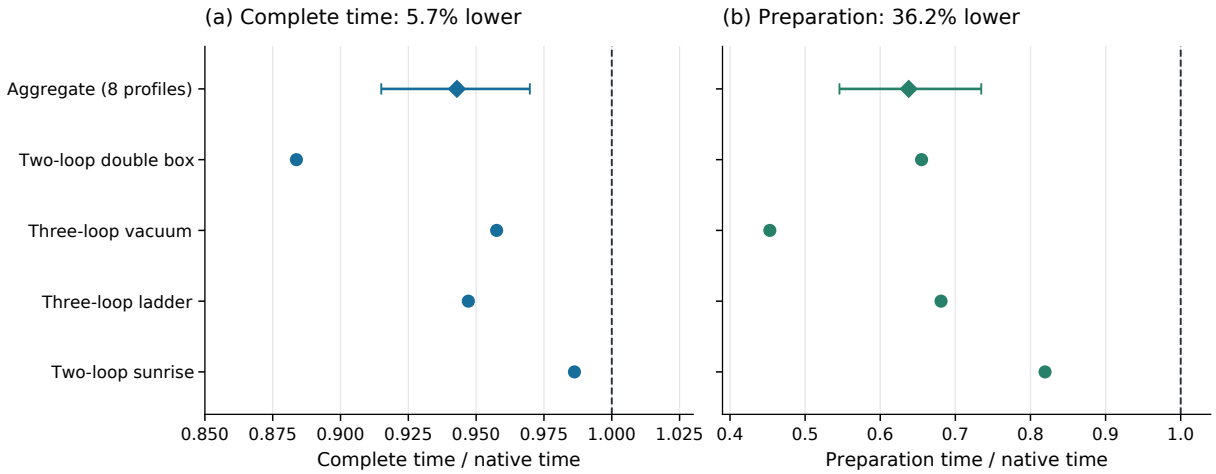


Figure 3: Equation selection saves more preparation time than complete reduction time. Each ratio compares the model with native choices in the same executable. The panels use different labelled ranges. The aggregate intervals describe variation across the eight profiles.

and a box with one external virtuality. Appendix F defines their propagators and invariants, using the massless limit of the nonplanar family in Ref. [4]. Four batches use rational kinematics and symbolic d. Three more keep the independent invariants symbolic, with coefficient fields $\mathbb{Q}(d, s, t)$, $\mathbb{Q}(d, s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$, and $\mathbb{Q}(d, s, t, v)$. Here s, t are four-point invariants, $s_{i,i+1}$ are adjacent five-point invariants, and v is an external squared momentum.

The seven batches contain 88 targets and produce seventy comparison calculations. All final results pass exact checks, but every one of the 35 filtered calculations needs native fallback. The average time ratio is 1.252, or 25.2% more time. Checks cover 5,600 coefficient pairs, agreement with enlarged seed regions in all seven cases, and agreement after new rational substitutions in the three symbolic cases. The fallback therefore restores a correct answer at an additional cost. Figure 4 records this transfer test before the new training described next.

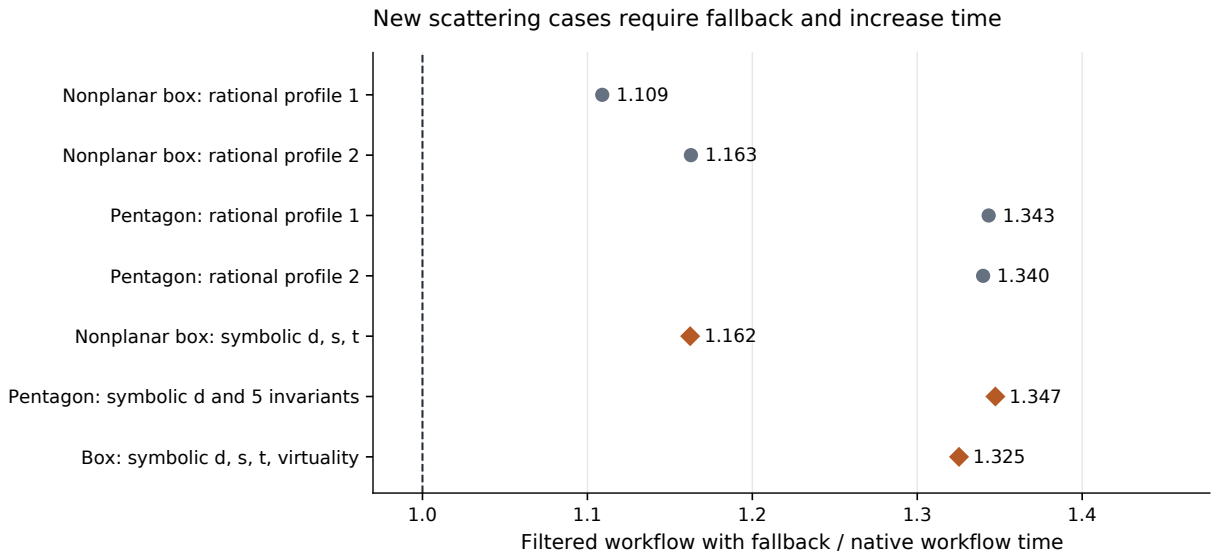


Figure 4: The original equation model is slower on the new scattering structures. Each point compares five-run median times, including fallback. All 35 filtered calculations require a retry. This historical test uses the original weights, before additional training.

4.4 Training on the additional scattering structures

The transfer failures raise a specific question: can examples from the missing structures teach a filter to retain enough equations? To test this, we train a new seed tree and equation tree for each of the nonplanar box, pentagon, and box with external virtuality. The original models remain fixed for comparison. Appendix C defines the new reference labels and tree fits.

Table 3 separates the training, validation, and test batches. Rational batches use different invariant values in each split, while symbolic batches keep the independent invariants symbolic and use disjoint target lists. The graph structures occur in all three splits. Thus the test measures learning within these structures and separates that question from transfer to an unseen graph.

Table 3: The additional scattering study. Each of the three structures contributes equally to the batch counts.

Part	Rational batches	Symbolic batches	Targets
Training	6	3	108
Validation	3	3	60
Testing	6	3	108

Each tree has maximum depth twelve. The seed tree uses sixteen features and gives all operators of a seed the same score; the equation tree also receives the operator index, giving seventeen features and allowing different decisions for different operators. In both cases Kira enumerates its usual seed region, the tree filters candidates before coefficient construction, and native elimination remains active. These fits produce new trees trained on native reductions, while the original neural networks remain unchanged.

The fits use 59,706 equation records and 4,674 grouped seed records, including candidate patterns repeated in different training batches. Table 4 gives the tree sizes. Validation compares three score thresholds and fixes a setting before the reserved test. The four test methods are native choices, the original equation tree, the new seed tree, and the new equation tree. Five repetitions on identical targets give 180 calculations, with prediction, exact algebra, coefficient checks, cleanup, and fallback included in the timer. Reference discovery, training, and later recording of results are excluded.

Table 4: Additional training records and tree sizes. Each graph uses three training batches.

Structure	Seed records	Seed nodes	Equation records	Equation nodes
Nonplanar box	4,326	67	56,238	819
Pentagon	258	15	2,838	179
Box with virtuality	90	11	630	81

All 180 calculations pass exact checks of 9,880 returned coefficient entries. Eighteen additional applications receive the target and prescribed basis without reference coefficients, and their answers also pass an independent comparison after reduction. Each new selector completes all 45 timed attempts without fallback, whereas the original equation model needs fallback in all 45 trials. The new training therefore improves sufficiency on these inputs.

The timing result is more limited. Relative to native choices, the new seed model has ratio 1.027 with descriptive 95% interval $[0.995, 1.086]$, and the new equation model has ratio 1.017 with interval $[0.988, 1.054]$. Their measured average times are 2.7% and 1.7% higher, respectively. The original filter has ratio 1.368, or 36.8% more time, on this new test. Its earlier 25.2% overhead refers to different parameter points and symbolic targets. The additional training removes much of the fallback cost but does not establish an overall saving against native Kira.

Table 5: New reserved test results. Each value uses five repetitions. Ratios divide model time by native time; values below one are faster.

Case	Native (s)	Original tree	New seeds	New equations
Nonplanar box (13)	4.163	1.293	1.023	0.980
Nonplanar box (14)	4.140	1.221	1.002	0.974
Nonplanar box (symbolic)	4.458	1.233	0.983	1.010
Pentagon (13)	1.005	1.439	0.963	1.016
Pentagon (14)	0.965	1.436	1.069	1.038
Pentagon (symbolic)	1.120	1.356	0.958	0.979
Box with virtuality (13)	0.826	1.339	0.953	0.991
Box with virtuality (14)	0.763	1.529	1.152	1.124
Box with virtuality (symbolic)	0.729	1.499	1.166	1.052
Geometric mean ratio	—	1.368	1.027	1.017

The individual results in Table 5 and Figures 5–6 explain this distinction. Some new filters reduce time, while others add overhead to cases that native Kira already solves quickly. We therefore retain native choices as the default for this scattering study. The result supports training on the missing structures to improve sufficiency, although it cannot isolate missing data as the sole cause of the original failure: the study also changes the seed-model architecture and fits separate trees for each graph. The intervals resample the three graph groups and describe this limited comparison.

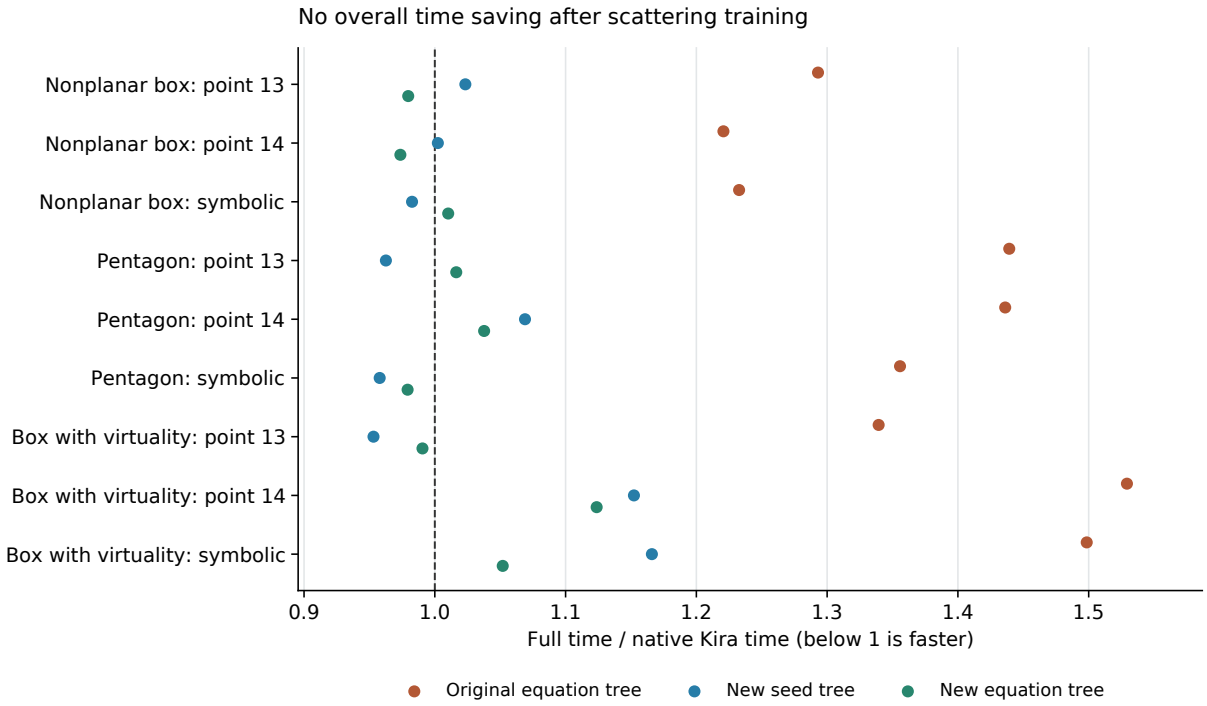


Figure 5: Training on the three structures removes much of the original filter's overhead, but gives no overall saving against native Kira. Points compare five-run median complete times, including prediction, checks and fallback. The dashed line marks equal time. No per-case uncertainty interval is inferred from five repetitions.

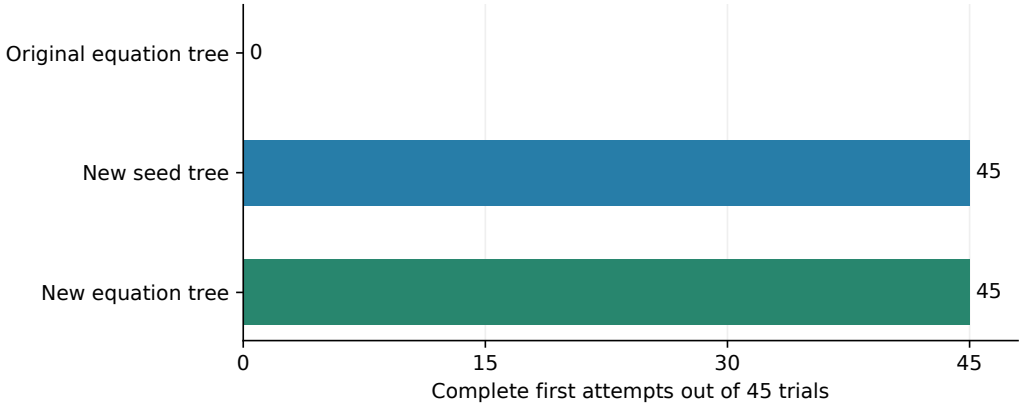


Figure 6: The new selectors provide sufficient equations on every first attempt. Each method has 45 trials across nine batches. Native fallback lets the original filter eventually complete, and its added time remains in the comparison above.

5 Selecting a complete calculation before equation generation

The local-model tests show that useful equations, accurate labels, and faster individual stages do not always lead to a faster complete reduction. We therefore change the training objective to measured calculation time and move the decision to the start of a target batch. The following studies examine the available strategies, the benefit of separate models for each graph, the treatment of seed expansion, and the number of workers used by Kira.

5.1 Learning from complete reduction time

The scattering test shows why a useful equation label is an incomplete training objective for acceleration. A filter can retain sufficient equations and still add more overhead than it saves. We therefore train a model to predict complete reduction time and let it choose the initial seed region, an optional seed or equation filter, and an integral ordering. The model makes one choice for the whole target batch before Kira begins its exact calculation.

To evaluate that choice fairly, we also need a baseline with the same surrounding implementation. We combine the earlier equation filter with an existing improvement in temporary data handling, which reduces execution cost without changing the equations or arithmetic. The *integrated native control* uses this implementation with Kira's own decisions. Comparing the learned strategy with that control measures the benefit of the prediction beyond the shared implementation changes.

For a target batch, let $B = (r_{\max}, s_{\max}, q_{\max})$ contain the largest positive index sum, numerator rank, and dot count. A dot is one denominator power above one. Table 6 lists the strategies considered, and Appendix I defines the quantities and derives the cost models. The seed limits and ordering options are provided by Kira [13]; ordering five changes the priority of sectors while retaining the within-sector preference of ordering one. All strategies must return the same prescribed master basis.

Table 6: Strategies available to the complete-time model. Negative seed bounds are replaced by zero. Native row choices remain active.

Strategy	Initial seed region and selection
Integrated native	Bounds B; native choices
Smaller region	Bounds $B - (1,1,1)$; native choices
Fewer dots	Bounds $B - (1,0,1)$; native choices
Equation filter	Bounds B; saved equation tree
Seed filter	Bounds B; saved seed tree where available
Different ordering	Bounds B; Kira ordering five
Larger region	Bounds $B + (1,1,1)$; native choices

A smaller region can avoid unnecessary equation generation, while a larger region can avoid a failed first attempt when the target's own index limits are insufficient. If the initial strategy fails the basis check, the calculation tries native choices at B and then at $B + (1,1,1)$ when needed. The native control also enlarges an insufficient region, and every failed attempt contributes to the training and test time. For the four earlier multiloop graphs, the seed-filter option uses native choices because this implementation has no separate seed tree for them. Its equation-filter option uses the earlier trees for those graphs and the newly trained trees for the three additional scattering structures.

The model receives eighteen features describing the graph, symbolic variables, number of targets, index powers, and sectors. For each strategy, its training label is the logarithm of the complete-time ratio to integrated native choices. We compare a constant predictor, six regression trees, three ridge-regression models, and two nearest-neighbour models. Regression trees follow

the squared-error construction of Ref. [12]; Appendix I specifies all candidates. Validation selects ridge regression with penalty $\lambda = 10$ and decision margin $\gamma = 0$, giving validation ratio 0.993.

The best fixed strategy selected from training is integrated native reduction. Consequently, the integrated-native and best-fixed controls use the same settings in independent trials, and differences between their times indicate execution variation. The seed and equation trees retain their earlier weights throughout this study, and learned row selection remains disabled.

The data contain 31 training batches, 17 validation batches, and 31 test batches from the sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality. Rational batches use different masses or invariants across the splits, while additional symbolic batches use different target lists. These are new inputs within seven known graph structures at one through three loops. The recurrence study through twelve loops remains a separate result for its restricted vacuum family.

Each training and validation cost uses three repetitions. Testing uses five repetitions to compare the preceding native implementation, integrated native choices, the best fixed strategy, and the learned strategy. The controller and models are loaded before timing, but every attempt starts a fresh Kira process. Timers include input preparation, prediction, native IBP generation, algebra, exact checks, cleanup, and retries. They exclude training, enlarged reference reductions, and later recording of results. Additional application checks measure cold startup and compare returned coefficients after a reduction supplied only with its targets and basis. For an unknown graph or target complexity outside the training range, the learned application uses native choices; the same guard applies to combined requests.

5.1.1 Results for individual target batches

All 620 timed calculations pass exact checks of 59,640 coefficient entries, and all 31 applications supplied without reference coefficients pass their subsequent comparisons. Each timed method makes fifteen fallback attempts. Thus this model changes some initial strategies but does not remove the remaining cases that need a second attempt.

Relative to the preceding native implementation, the learned strategy has time ratio 0.9903, with descriptive interval $[0.9796, 0.9996]$, or about 1.0% less time. Its ratio to integrated native choices is 0.9965 with interval $[0.9875, 1.0070]$, and its ratio to the best-fixed control is 1.0026 with interval $[0.9979, 1.0081]$. Both comparisons with the matched integrated controls include equal time. They therefore provide no reliable additional saving from the learned choices. The two identically configured controls have ratios 0.9938 and 0.9878 to the preceding native implementation, illustrating the scale of execution variation.

Table 7 reports the results by graph. The aggregate averages logarithmic ratios within each graph and then gives all seven graphs equal weight, as derived in Appendix I. The smallest learned-to-native ratio is 0.9654 for the three-loop vacuum, while the ladder and nonplanar box take slightly more time. These results motivate testing more specific strategies for each graph rather than enabling the common model for all inputs.

Table 7: Complete-cost results. Native seconds are geometric means of batch median times. The other entries are geometric mean time ratios; a value below one means less time.

Graph	Native (s)	Integrated/native	Learned/native	Learned/integrated
Sunrise	0.502	1.0123	0.9932	0.9811
Planar box	1.621	0.9906	0.9771	0.9864
Three-loop vacuum	0.726	0.9745	0.9654	0.9907
Three-loop ladder	3.535	1.0146	1.0013	0.9869
Nonplanar box	3.119	1.0075	1.0073	0.9998
Pentagon	0.599	0.9758	0.9942	1.0189
Box with virtuality	0.484	0.9824	0.9944	1.0122

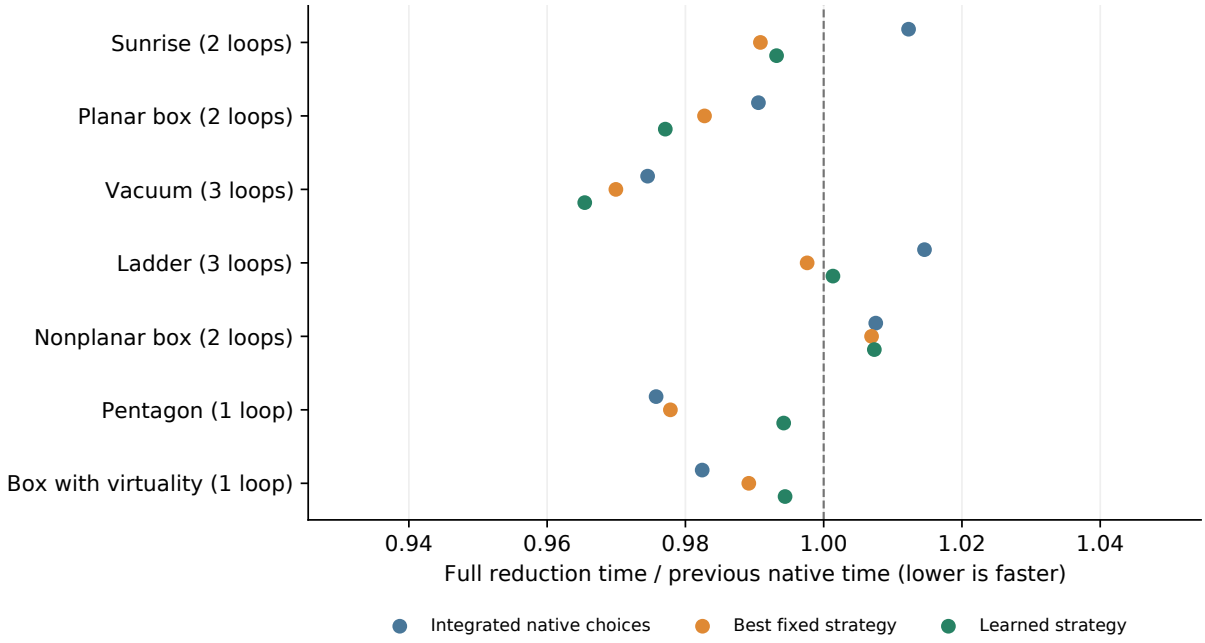


Figure 7: Learned choices and the native controls have similar complete times. The best-fixed and integrated controls use the same configuration in separate trials. The plot shows variation across graph structures; the text gives the aggregate uncertainty intervals. The axis focuses on the measured range and retains the equal-time line.

5.1.2 Sharing a reduction between compatible requests

Before changing the predictor, we test whether repeated work can be removed from the way Kira receives targets. When two available requests have the same family and kinematics, their target lists can be combined and reduced together. The resulting coefficient rows are then returned to the appropriate request. This shares family preparation and algebra between the lists. Appendix I defines the ten request groups and derives this separation from linearity; the model and its settings remain fixed.

Five repetitions of four methods give 200 complete calculations, all of which pass exact checks. Joint native reduction has ratio 0.5252 to two separate native reductions. With the integrated implementation and native choices, the ratio is 0.5175 with interval $[0.5073, 0.5346]$, giving 48.3% less time or speedup 1.93 with interval $[1.87, 1.97]$. Every tested graph group benefits from the shared reduction. Its integrated-native ratios range from 0.5040 to 0.5679, corresponding to savings of 43.2–49.6%.

The joint learned strategy has ratio 0.5152 to separate native requests, but its ratio to joint integrated native choices is 0.9955 with interval $[0.9810, 1.0058]$. As Table 8 and Figure 8 show, the main gain comes from sharing the calculation; the predictor adds no reliable improvement over the matched joint control. The standard application therefore uses the fixed strategy selected from training, which retains native choices. Ten checks of each joint application and ten checks of the single-batch application supply only a basis and verify the returned coefficients independently, in addition to the 31 application checks above.

Table 8: Joint-request time ratios and descriptive intervals over the seven graph groups. Each group contains two compatible target requests.

Comparison	Time ratio	95% interval
Joint native / separate native	0.5252	$[0.5130, 0.5420]$
Joint integrated / separate native	0.5175	$[0.5073, 0.5346]$
Joint learned / separate native	0.5152	$[0.5022, 0.5322]$
Joint learned / joint native	0.9810	$[0.9657, 0.9946]$
Joint learned / joint integrated	0.9955	$[0.9810, 1.0058]$

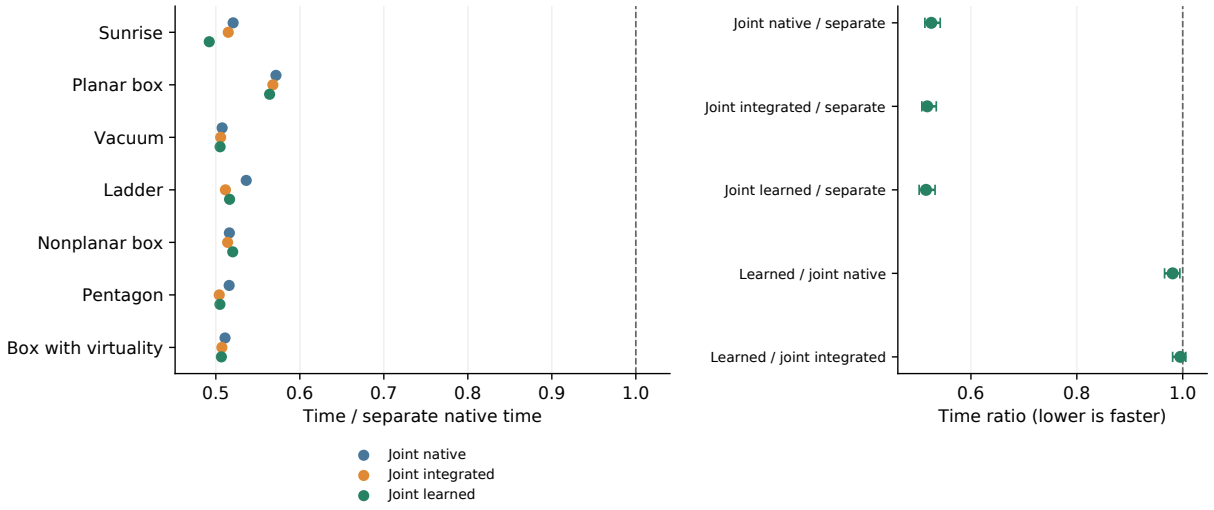


Figure 8: Combining compatible requests gives the main saving. Left: ratios by graph relative to two separate native reductions. Right: aggregate ratios and descriptive intervals. Comparing learned choices with joint integrated native choices gives no reliable extra saving. The panels retain the equal-time reference.

The joint comparison reuses the main test's reserved parameter points, so it provides an additional comparison on that population. Its saving requires requests that can share one calculation. Transfer to new graph structures, isolated arbitrary integrals, and general scattering beyond three loops remains outside this result. The next study returns to individual batches and tests whether separate models and a wider set of seed strategies give a measurable gain.

5.2 Choosing a strategy separately for each graph

A common predictor may miss differences in the work required by different graph structures. We therefore fit separate predictors for the seven graphs and enlarge the available seed strategies. The added choices can restrict seeding to maximal target sectors or reduce numerator seeding in lower sectors through Kira's truncation option [5]. Each method first enforces a valid positive-index bound for its declared sectors, and the final basis check and fallback remain in place.

For each graph, ten candidate models predict the complete-time ratios of nine strategies from nine target and family features. The candidates include regression trees, a bagged ensemble of trees, regularized linear models, and nearest-neighbour models. Their cost labels include unsuccessful attempts and retries. The data contain 36 training batches, 18 validation batches, and 36 reserved test batches at new rational parameter values, with d symbolic throughout. Targets include propagator powers up to three, lower sectors, and auxiliary numerator indices -2. Appendix J gives the exact input definitions and selection procedure.

Training and validation contain 2,798.5 seconds of measured reductions, while fitting all seventy candidates takes 0.86 seconds. These offline costs exclude reference construction, development, and replaced measurements and are not included in application time. Validation enables a new predictor only if it saves at least five percent against the fastest available control. The controls are native choices, a deterministic sector-and-truncation rule, and a fixed strategy for each graph selected from training; the fixed strategy may itself use the older equation filter. This requirement activates predictors for three graphs and retains a control for the others. Unknown graphs and features outside the training range use native choices.

Five repetitions of four methods give 720 test calculations, all of which pass exact coefficient checks. With equal weight for each graph, the selected method has ratio 0.9368 to native choices, with descriptive interval $[0.8867, 0.9878]$, corresponding to 6.3% less time. Its ratio to a fixed strategy for each graph is 0.9441 with interval $[0.8992, 0.9863]$. This second comparison isolates the benefit of selecting a strategy from the targets beyond choosing one setting for the whole graph.

Table 9 shows lower time for five of the seven groups. The enabled trees save 14.2% for the planar box, 12.4% for the ladder, and 13.8% for the nonplanar box. The vacuum and box with external virtuality take 3.7% and 0.8% more time, respectively, while retaining control strategies throughout. In particular, the vacuum uses native settings, so its difference includes the extra selection overhead and execution variation. Across 180 timed trials, the selected method needs no retries and native choices require 40; these costs are included in the ratios.

Table 9: Reduction time with a separate strategy for each graph. Times are geometric means of batch medians in seconds, and the ratio divides selected time by native time. The strategy column identifies the enabled model or retained control.

Graph	Strategy	Native	Selected	Ratio
Sunrise	Stronger truncation	0.825	0.811	0.983
Planar box	Depth-two tree	3.410	2.925	0.858
Vacuum	Native	1.033	1.072	1.037
Ladder	Depth-one tree	5.144	4.506	0.876
Nonplanar box	Depth-two tree	6.234	5.376	0.862
Pentagon	Target sectors	1.040	0.989	0.951
Virtuality box	Sectors and truncation	0.808	0.815	1.008

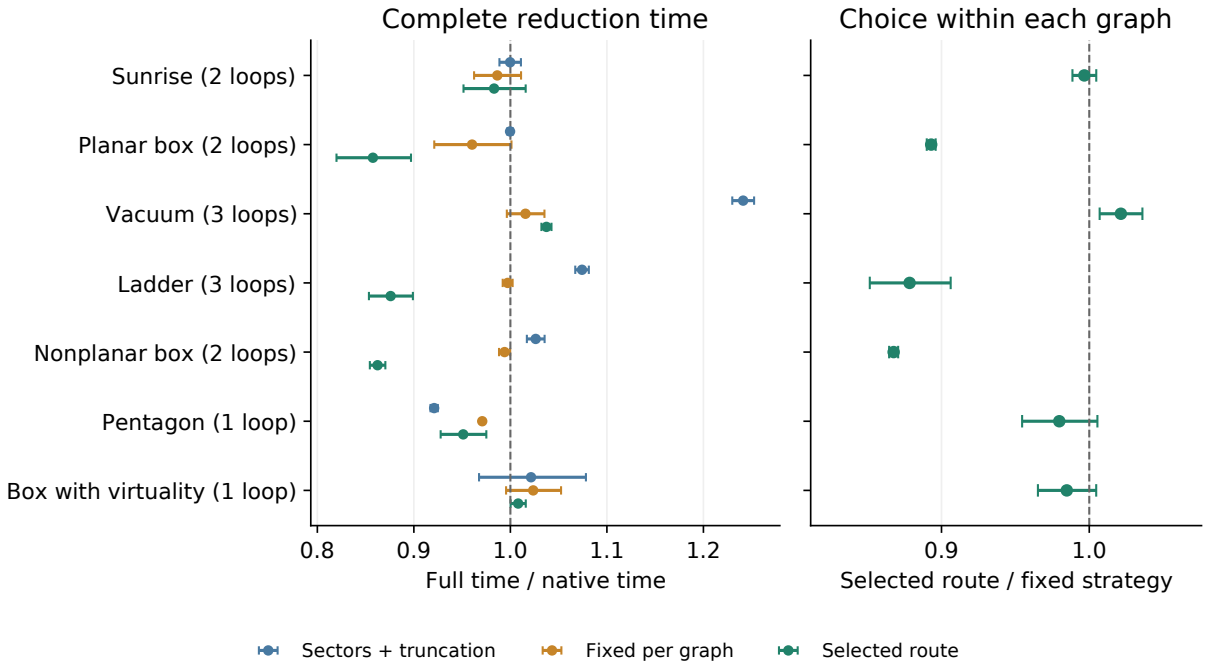


Figure 9: The effect of choosing a separate policy for each graph. Left: complete-time ratios to native choices. Right: the selected method relative to one fixed strategy per graph. Bars show the range across two reserved parameter profiles; they are not significance intervals. Both panels retain the equal-time reference.

The timed trials check 14,040 coefficient entries. A further 36 applications supplied only with the master basis and seven additional application checks verify 945 coefficients independently. These checks establish correct completion for the measured inputs. The graph structures and target patterns are known during training, and each graph has only two training parameter profiles. The reserved profiles therefore test new kinematic values within these inputs, leaving new target structures and general scattering beyond three loops for further study.

5.3 Expanding seeds selectively and using an intermediate basis

The graph-specific study identifies symbolic equation regeneration as the largest measured solver stage. Some target batches need seed limits beyond those of the targets themselves, and a general enlargement can generate many unnecessary equations. We therefore test whether selective increases in the limits, or a reduction through an intermediate master basis, can lower this remaining cost.

The candidate set contains six ways to increase selected seed limits and two ways to reduce first to a temporary basis. In the latter case, Kira reduces the remaining temporary masters to the prescribed basis, and exact multiplication combines the two coefficient matrices. The final basis requirement is the same for every method. Appendix K defines the candidates and derives this conversion, which allows the first target reduction to use a smaller system when the remaining basis conversion is inexpensive.

For development batch c, let $B_c = (r_c, s_c, q_c)$ denote valid initial limits on the positive index sum, numerator rank, and number of extra denominator powers. We retain the fastest new candidate for a development batch only if it saves at least ten percent against the graph-specific method in Section 5.2. The resulting candidate set contains that preceding method, full-sector basis completion, and the selective increments $B_c + (1, 0, 0)$, $B_c + (0, 1, 0)$, and $B_c + (1, 1, 0)$. Development results select the options but do not enter the reserved comparison.

The new data contain 36 training batches, 18 validation batches, and 36 test batches from the same seven graphs and target patterns, with new rational kinematics and symbolic d. Ten predictors per graph use the same nine features to predict time relative to the preceding method. Validation compares each predictor with that method and with a fixed new strategy selected from training for each graph. A predictor is enabled only if it saves at least five percent against the faster control; otherwise that control is used. A fixed candidate may itself call the preceding learned method, so this comparison tests the additional choice among the new options.

Validation enables new predictors for three graphs. The retained training and validation records contain 1,731.7 seconds of measured reductions, and fitting the seventy candidates takes 0.88 seconds. These costs exclude reference construction and development and remain outside the reserved reduction timer.

The test compares native choices, the preceding graph-specific method, the new fixed strategy, and the selected strategy. Five repetitions give 720 calculations, all of which pass exact checks. A further 43 applications also pass when the reduction receives no target reference coefficients. With equal weight for each graph, the selected method has ratio 0.9115 to the preceding method, with descriptive interval $[0.8105, 0.9980]$, giving 8.8% less time. Its ratio to native choices in the same experiment is 0.8516 with interval $[0.7282, 0.9647]$, giving 14.8% less time. Relative to the new fixed strategy, the ratio is 0.9264 with interval $[0.8760, 0.9747]$.

The sum of batch medians answers a different question: how long would this set of requests take when each is counted once? For the 36 reserved batches, this sum is 106.24 seconds for native choices, 96.32 seconds for the preceding method, and 77.03 seconds for the selected method. The geometric ratios above instead give every graph equal weight. Both measures show a saving, although their magnitudes differ because the longer calculations contribute more to the workload total.

The stage measurements explain where part of the saving occurs. Across 180 runs per method, symbolic equation regeneration falls from 138.02 to 80.58 seconds. Neither the preceding nor the selected method makes a fallback attempt, compared with 40 for native choices. Planned second-stage reductions of temporary masters are included in complete time and are counted separately from fallback. The comparison with the fixed strategy supports an additional benefit from the selection procedure, which combines validation-based control choices with predictions for individual target batches.

Table 10: Selective seed expansion compared with the graph-specific method in Section 5.2. Times are geometric means of batch medians in seconds. "Previous" denotes that graph-specific method and "New" denotes the seed-expansion method. Each graph has two reserved parameter profiles.

Graph	Policy	Previous (s)	New (s)	New/previous	New/native
Sunrise	Add s	0.596	0.593	0.995	0.979
Planar box	Tree depth 1	2.422	2.062	0.851	0.735
Vacuum	Complete basis	0.928	0.941	1.013	1.016
Ladder	Tree depth 1	4.477	3.007	0.672	0.584
Nonplanar box	Tree depth 1	4.091	3.550	0.868	0.759
Pentagon	Previous	0.723	0.754	1.043	0.998
Virtuality box	Previous	0.580	0.582	1.003	1.004

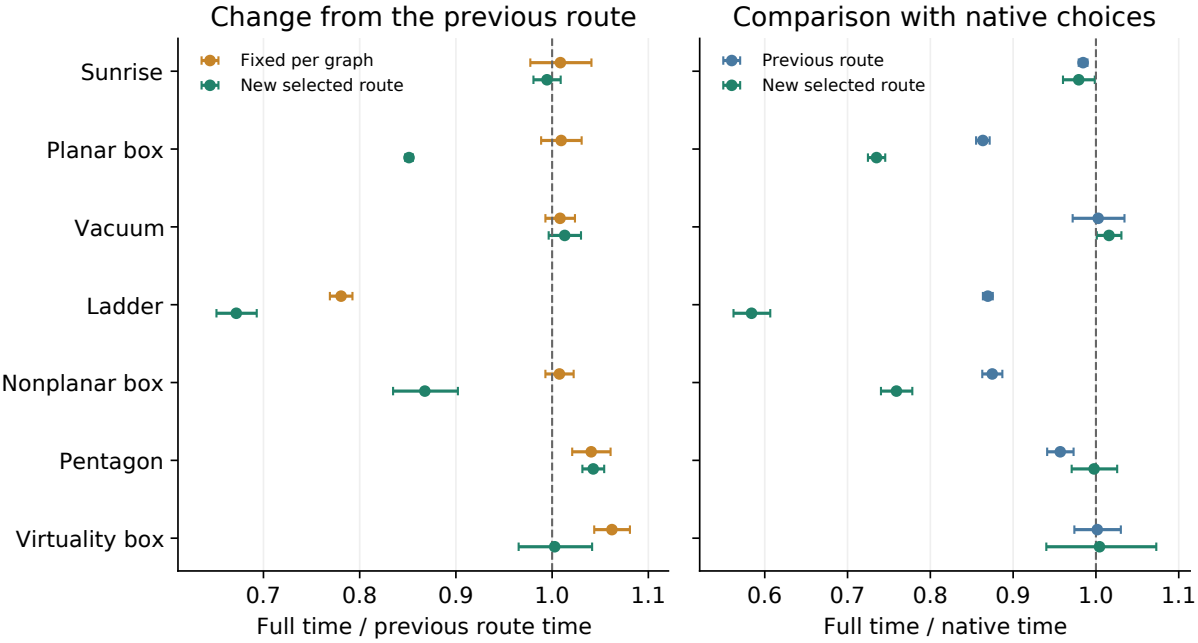


Figure 10: Selective choices change complete reduction time. Left: the new fixed strategy and selected method relative to the preceding method. Right: the preceding and new methods relative to native choices in the same test. Values below one are faster. Bars show the range over the two reserved profiles; they are not significance intervals.

The gain is not uniform across the graphs. Relative to the preceding method, the vacuum takes 1.3% more time, the pentagon 4.3% more, and the box with external virtuality 0.3% more. All reported times include prediction, native IBP generation, basis conversion, exact algebra, checks, cleanup, and fallback. These results measure the additional method directly on new parameter profiles; they do not multiply gains from earlier experiments. Their scope remains known graphs and target patterns with rational kinematics, rather than unrestricted symbolic kinematics or arbitrary high-loop scattering.

5.4 Testing integral and propagator orderings

Selective seed expansion reduces equation-generation work, but the ordering of the resulting integrals can still affect elimination. Kira provides eight integral priority rules and four automatic propagator-ordering schemes [13, 5]. We test whether a model can choose among these settings from the target indices before equation generation, while retaining the seed strategy from Section 5.3. The model makes one decision for each batch, and a failed basis check triggers a repeat with the preceding settings. Appendix L defines the candidates and shows why consistent relabelling preserves the integrals.

Development first compares eleven candidates and then tests the pair of best nondefault orderings for each batch. We keep the preceding settings and any batch winner that saves at least ten percent of complete time. If ω is the integral-order number and χ the propagator-scheme number, the retained new settings are $(\omega, \chi) = (2, 1)$, $(7, 1)$, $(5, 4)$, and $(8, 3)$. The preceding method also remains available without a change in its settings.

We fit ten predictors per graph on 36 training batches, select their use on 18 validation batches, and test on 36 reserved batches. The features, complete-time labels, candidate model types, and five-percent validation requirement are the same as in the seed-expansion study. A fixed ordering still uses the preceding learned seed strategy, so the fixed-setting comparison measures the additional ordering decision. Training and validation contain 1,432.8 seconds of reductions, and fitting the seventy predictors takes 0.93 seconds. These offline costs exclude development and reference construction.

None of the predictors passes the validation requirement. The tested selected method therefore uses fixed ordering settings chosen by validation, with no newly enabled ordering predictor. Any difference in its time reflects those settings, their interaction with the retained seed strategy, and execution variation. This distinction is necessary when interpreting the results as evidence about machine learning.

All 720 timed reductions and 43 additional applications pass exact checks; the additional applications receive no target reference coefficients during reduction. With equal weight for each graph, the selected-to-preceding ratio is 1.0141 with descriptive interval $[0.9986, 1.0303]$, giving 1.4% more time. The selected-to-native ratio is 0.8499 with interval $[0.7306, 0.9642]$, giving 15.0% less time, while the ratio to the new fixed strategy is 1.0070 with interval $[0.9811, 1.0327]$. The saving against native choices therefore does not show an improvement over the already learned seed-expansion method.

The sum of median times over the 36 test batches is 110.09 seconds for native choices, 79.61 seconds for the preceding method, 79.53 seconds for the new fixed settings, and 79.68 seconds for the selected settings. On this workload, the selected method takes 0.1% more time than its predecessor. Its additional ordering decision takes a median of 271.1 microseconds per application, and both this cost and the retained seed-policy cost are included in complete time.

Table 11: Ordering settings compared with the seed-expansion method in Section 5.3. Times are geometric means of batch medians in seconds. "Previous" retains the earlier ordering, and "New" uses the selected setting. The pair (ω, χ) specifies the integral order and propagator scheme.

Graph	Choice	Previous (s)	New (s)	New/previous	New/native
Sunrise	$(7, 1)$	0.610	0.633	1.037	1.034
Planar box	$(2, 1)$	2.407	2.390	0.993	0.759
Vacuum	Previous	0.975	0.982	1.007	0.983
Ladder	Previous	3.122	3.088	0.989	0.591
Nonplanar box	$(2, 1)$	3.698	3.681	0.995	0.745
Pentagon	$(2, 1)$	0.788	0.820	1.041	0.987
Virtuality box	$(5, 4)$	0.601	0.624	1.039	0.955

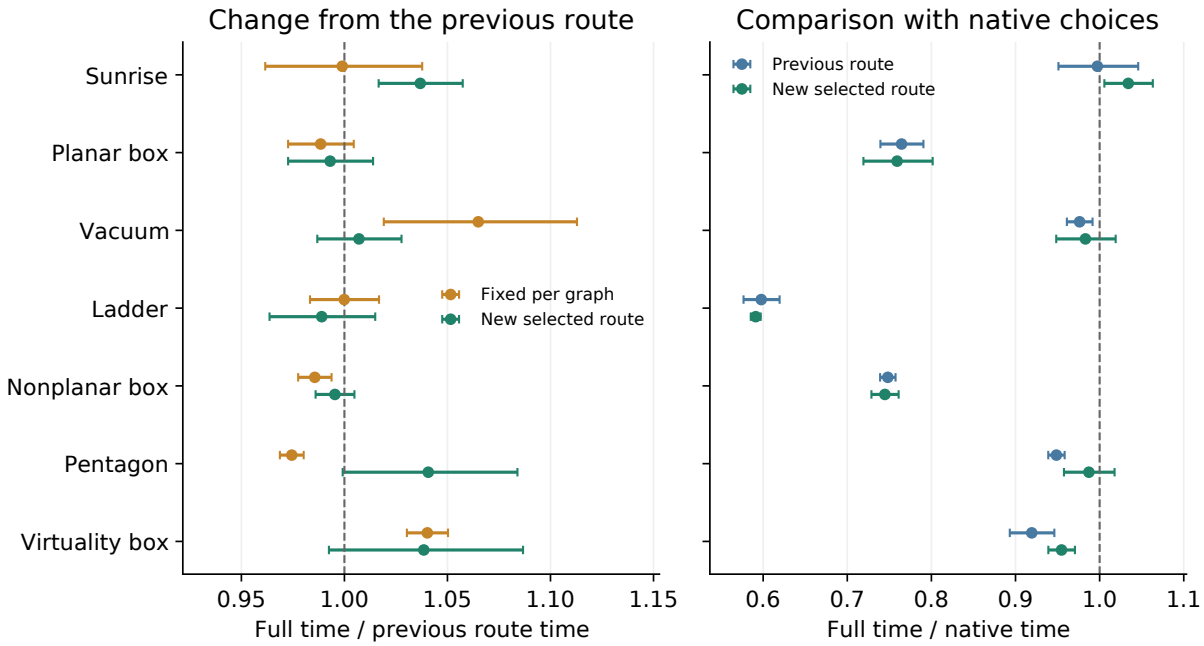


Figure 11: The effect of choosing Kira settings before reduction. Left: new fixed settings and selected settings relative to the preceding method. Right: preceding and selected methods relative to native choices. Values below one indicate less complete time. Bars give the range across two parameter profiles; they are not significance intervals.

The sunrise, vacuum, pentagon, and box with external virtuality take 3.7%, 0.7%, 4.1%, and 3.9% more time, respectively. No selected calculation requires fallback. Across 180 runs per method, symbolic equation regeneration changes from 79.88 to 80.58 seconds, while back-substitution changes from 37.99 to 37.35 seconds. These stage measurements show small changes in opposite directions rather than a consistent overall improvement.

The timers include prediction, preparation, native generation, exact checks, basis conversion, retries, and cleanup. As in the preceding comparisons, the caller and models are already loaded, and each attempt starts a fresh Kira process. The evidence therefore supports retaining the earlier ordering for this measured workload of known one- through three-loop graphs at new rational parameter values. The next comparison keeps that choice and changes the amount of parallel work instead.

5.5 Choosing how many Kira workers to use

The ordering study gives no further saving, so we retain the seed-expansion method and its earlier ordering. We next test the number of Kira workers, which determines how much of the symbolic calculation can run in parallel [4]. Development measurements show that extra workers add overhead on small batches, while some larger box batches finish sooner. This suggests choosing the worker count from the requested targets before equation generation.

The candidate counts are one, two, and four workers in the same integrated executable. The native control uses Kira's own seed and reduction decisions together with the common earlier implementation changes. For the learned comparison, the nine target features used above predict each worker count's complete time relative to one worker. We fit ten predictor types per graph on 36 training batches, select settings on 18 validation batches, and reserve 36 more batches for testing. Validation minimizes the sum of batch median times. A predictor must save at least two percent against the faster control and give lower summed time in at least two of three validation repetitions; otherwise the control remains active. Appendix M gives the labels, checks,

and comparison rules.

Validation enables a depth-two tree for the planar box and a depth-one tree for the nonplanar box. The other five graph types retain the preceding one-worker strategy. Both trees split only on the number of targets in the batch and choose at most two workers. Since the tested target patterns repeat across the parameter splits, these results establish a useful decision within those patterns; they do not establish target count as a sufficient description of arbitrary integral structures. Training and validation contain 1,098.1 seconds of reductions, and fitting all seventy candidates takes 0.82 seconds. These offline costs exclude reference construction and development.

All 900 timed reductions and 50 additional applications pass exact checks. The additional applications receive targets and a prescribed basis without reference coefficients. Seven run consecutively through a caller that keeps the models loaded, in a separate two-core allocation; each native attempt still starts a fresh Kira process. The timing experiment reserves four cores for every method so that the always-four-worker control can use its full setting. Thus every timed method has the same available resources, even when it uses fewer workers.

For the 36 reserved batches, the sum of median complete times falls from 82.21 seconds with the preceding one-worker method to 78.73 seconds with worker selection, a reduction of 4.2%. In the same experiment, native one-worker choices take 110.67 seconds, always using four workers takes 156.95 seconds, and a fixed count for each graph takes 82.45 seconds. Selection therefore gives 4.5% less time than the fixed-per-graph control. Table 12 shows where these differences arise; changes for the five graphs that retain one worker include execution variation and the additional decision cost.

Table 12: Time with and without worker selection. Entries sum the batch median times within each graph, in seconds. “Previous” uses the seed-expansion method with one worker; “Selected” uses the validation-selected count. The final column divides selected time by previous time.

Graph	Choice	Previous (s)	Selected (s)	Selected/previous
Sunrise	1 worker	5.271	5.215	0.989
Planar box	Tree depth 2	15.643	14.997	0.959
Vacuum	1 worker	4.813	4.620	0.960
Ladder	1 worker	23.197	23.602	1.017
Nonplanar box	Tree depth 1	26.768	23.819	0.890
Pentagon	1 worker	3.558	3.523	0.990
Virtuality box	1 worker	2.963	2.955	0.997

Giving each graph equal weight yields a smaller selected-to-preceding ratio of 0.9817, with descriptive interval $[0.9682, 0.9946]$, or 1.8% less time. The selected-to-native ratio is 0.8456 with interval $[0.7346, 0.9502]$, or 15.4% less time. These averages give equal importance to each graph, whereas the sum of medians measures the time for this particular collection of requests. Figure 12 also shows why simply increasing every calculation to four workers is ineffective on these small cases.

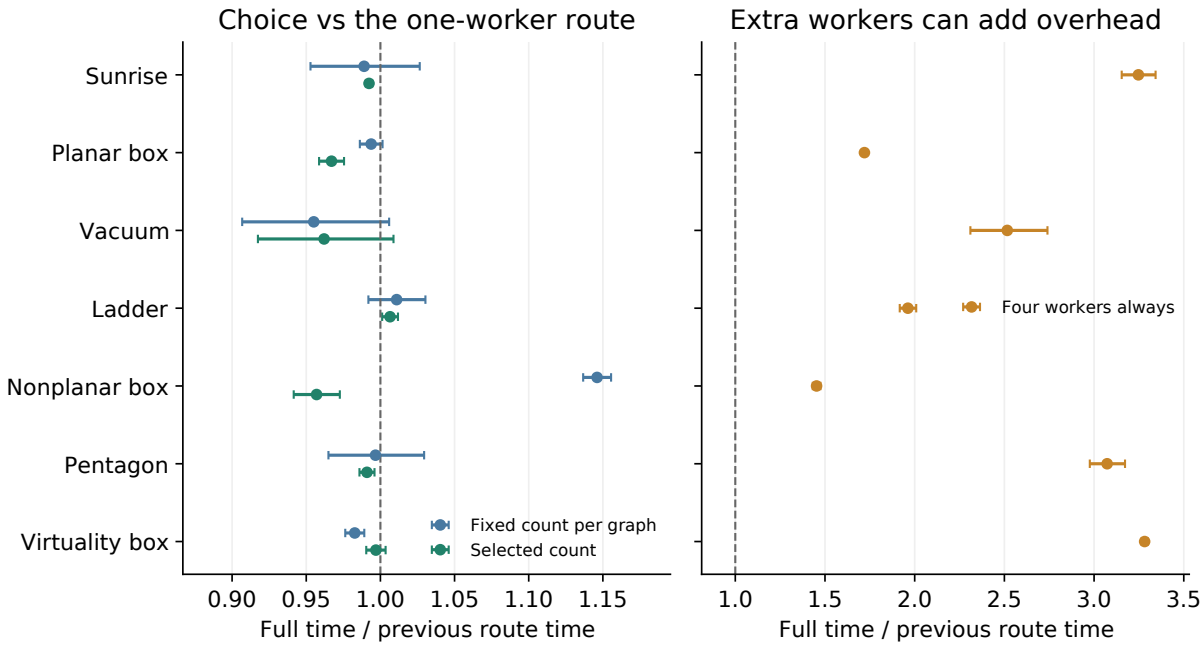


Figure 12: The effect of selecting the worker count. Left: selected and fixed worker counts compared with the preceding one-worker method. Right: always using four workers adds overhead on many small cases. Each panel has its own horizontal scale. Each point is the geometric mean of batch time ratios for one graph. Bars show the range across two parameter profiles and are not significance intervals.

Variation between repetitions matters for the size of the gain. Across all five repetitions, the selected method takes 404.84 seconds and the preceding method 414.08 seconds, giving 2.2% less elapsed time. Selection has lower summed time in four of the five repetitions, with selected-to-preceding ratios ranging from 0.9346 to 1.0555. This range describes the observed repetitions and is not a confidence interval. The ladder's sum of median times increases by 1.7%.

The saving in elapsed time also uses more CPU time. The extra prediction takes a median of 276.6 microseconds and is included in the complete-time measurements. Across 180 reductions per method, the preceding strategy uses 385.9 CPU-seconds, the always-four-worker strategy 1024.1, and the selected strategy 458.9. Selection therefore increases CPU use by 18.9%. These values measure CPU time consumed by the calculation, rather than the number of cores allocated to it.

The selected method has no failed attempts and requires no fallback. The always-four-worker method has ten unsuccessful native attempts, whose subsequent retries succeed; their costs remain in the reported time. Some native attempts report allocation failures under the imposed process limits. All comparisons include prediction, preparation, exact algebra, basis conversion, checks, retries, and cleanup, with the caller and models loaded before timing and a fresh Kira process for each attempt.

Worker selection is retained for this measured workload because it lowers elapsed time while preserving the exact results. Its demonstrated scope is seven known graph structures at one through three loops, known target patterns, and new rational kinematics with symbolic dimension. The preceding one-worker method remains available when the resource conditions or inputs differ. The result identifies a measured improvement among the tested choices, with the CPU cost stated alongside it.

6 Overall performance and physical scope

6.1 What the model comparisons establish

Table 13 brings together the separate experiments. The original seed network saves time for single-target requests from existing equation pools, and the equation tree saves time in its four trained native graph structures. Learned row selection gives no reliable saving and is omitted. Training on the additional scattering structures improves equation sufficiency, but the complete-time comparisons in Section 4.4 still favour native choices as the default for that study.

The later experiments assess how much a learned choice adds to an already integrated calculation. The first complete-strategy model gives no reliable gain over its matched native controls. Separate models for each graph then give 6.3% less time relative to native choices and ratio 0.9441 relative to a fixed strategy for each graph. Selective seed expansion improves that method further on new profiles, with 8.8% less time than its preceding method and 14.8% less time than native choices. The ordering comparison gives 1.4% more time than the seed-expansion method and therefore does not replace it. Worker selection finally reduces the sum of median times by 4.2%, while the saving across all repetitions is 2.2% and CPU use increases by 18.9%.

These results use different targets and comparison sets, so their percentage savings cannot be multiplied. The same distinction applies to combining requests: its 48.3% saving measures two compatible requests reduced together, and the cost model adds no reliable improvement over that shared native calculation. Each reported gain should therefore be read with the baseline and workload shown beside it.

Table 13: Summary of separate comparisons. Each row uses its stated inputs and baseline. The studies provide no measured product of these speedups.

Method	Tested calculation	Result for complete time
Seed network	One target from an existing equation pool	8.6% less time
Equation tree	Native reduction in four trained graph structures	5.7% less time
Row policy	Native symbolic elimination	No reliable saving
Original equation tree	Three new scattering structures	25.2% more time
Complete-cost model	Seven known graph structures, loops 1–3	No reliable gain over native controls
Policies per graph	Higher powers and lower sectors, loops 1–3	6.3% less time
Selective seed expansion	Known graphs, new profiles, loops 1–3	14.8% less time than native
Selecting Kira workers	Known graphs, new profiles, loops 1–3	4.2% less time than previous, sum of medians
Choosing Kira settings	Known graphs, new profiles, loops 1–3	No added gain over preceding method
Joint integrated reduction	Two compatible requests in seven graph structures	48.3% less time
Exact recurrences and reuse	Restricted vacuum family, loops 1–12	543.8× faster

6.2 Exact reuse in the restricted vacuum family

The high-loop vacuum study explores a different source of acceleration. For the family in Appendix D, exact recurrences express the reduction coefficients through $z = L(d-2)$. A coefficient calculation can then serve several loop orders and several targets. The method reuses interme-

diate coefficients and checks each result with an independent Taylor expansion; Kira remains available if this direct check fails. The large saving in this study includes mathematical reuse and implementation changes, so it is not an isolated measure of machine learning performance.

The comparison uses eight fresh groups, each containing three mass profiles with two, three, or four massive denominators, and requests loop orders one through twelve. Five repetitions compare the saved original Kira 3.1 executable, a later full-pool control, and the recurrence method on identical requests. All 120 calculations pass exact checks. The final-to-original time ratio is 0.001839, corresponding to speedup 543.815 with interval $[488.134, 605.064]$. A separate check verifies 27,960 nonzero target–master coefficient polynomials. The successful recurrence calculations need no Kira calls, whereas each baseline makes 1,440 calls across its trials.

This result depends on the structure of the family. The benchmark constructs Euler and mass partial-fraction identities rather than a general high-loop IBP system. Earlier checks at loop orders 13, 16, 24, and 32 use the same family, and the recurrence proof applies to every positive integer loop order under its stated assumptions. The demonstrated high-loop result therefore concerns this restricted vacuum problem; general high-loop scattering requires a separate comparison.

7 Conclusion

We have tested whether learned decisions can reduce the time required for exact Feynman integral reduction with Kira. The results support seed and equation selection in specific domains, while the row-selection experiments give no reliable saving. We therefore keep Kira's row choices and use learned decisions where the measurements show that they avoid enough work to justify their cost.

The later models make one decision before equation generation, selecting among complete strategies whose training labels include unsuccessful attempts and retries. Separate models for each graph and selective seed expansion improve the tested native reductions. Changing the integral ordering gives no further gain. Selecting the number of Kira workers reduces the sum of batch median times from 82.21 to 78.73 seconds on 36 reserved batches, compared with 110.67 seconds for native one-worker choices in the same experiment. Across all repetitions, the saving over the preceding method is 2.2%, with an 18.9% increase in CPU use. All 900 timed reductions and 50 additional applications pass exact checks.

The way requests are presented to Kira also affects the result. Combining compatible requests saves 48.3% in a separate comparison, and exact recurrence reuse gives a 543.8-fold speedup for the restricted vacuum family through twelve loops. These gains arise from different calculations and cannot be combined into a single cumulative speedup.

The final native tests cover seven known graph structures at one through three loops, using known target patterns and new rational kinematics with symbolic dimension. Within this domain, the study identifies useful learned choices and records their costs and slower cases. Establishing comparable gains for new target structures, general high-loop scattering, or unrestricted symbolic kinematics remains further work. The measured strategies also provide no proof of globally optimal reduction paths.

A IBP relations and exact elimination

This appendix derives the identities and row operations used throughout the study. For a seed $\boldsymbol{\sigma}$, write its denominator product as $P_{\boldsymbol{\sigma}} = \prod_i D_i^{-\sigma_i}$ and define $\partial_k^\mu = \partial/\partial \ell_{k\mu}$. The product rule separates the derivative in Eq. (2) into a derivative of the momentum vector and a derivative of

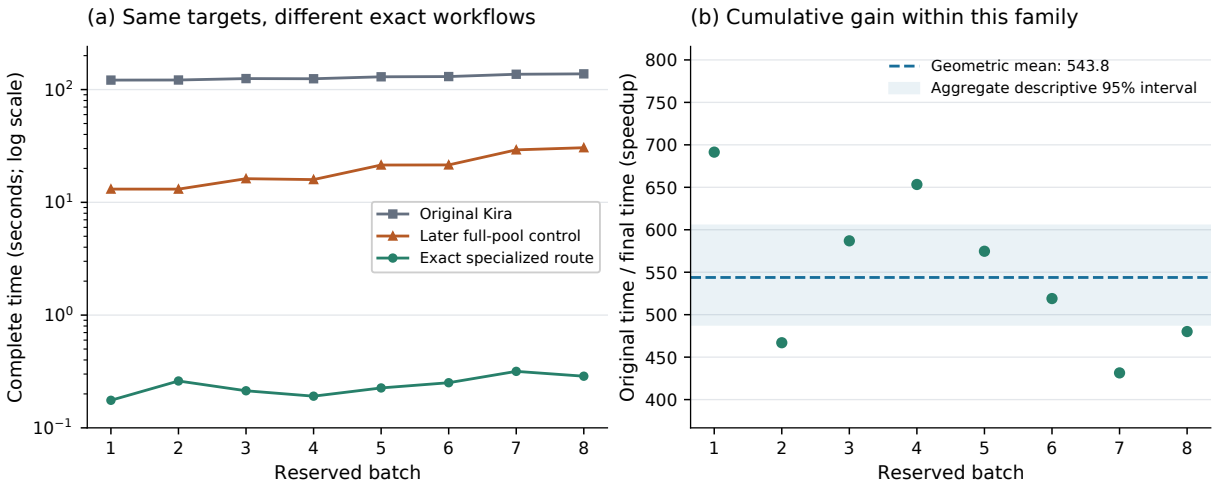


Figure 13: Exact recurrence reuse gives the large saving in the restricted vacuum family. Left: complete times on a logarithmic axis. Right: original time divided by final time. Each batch contains three mass profiles at loops 1–12. The result combines mathematical reuse and program improvements.

the propagator factors:

$$\partial_k^\mu (v_\mu P_{\boldsymbol{\sigma}}) = (\partial_k \cdot v) P_{\boldsymbol{\sigma}} + v \cdot \partial_k P_{\boldsymbol{\sigma}}, \tag{6}$$

$$v \cdot \partial_k P_{\boldsymbol{\sigma}} = -\sum_i \sigma_i (v \cdot \partial_k D_i) D_i^{-\sigma_i - 1} \prod_{j \neq i} D_j^{-\sigma_j}. \tag{7}$$

When $v = \ell_h$, the first derivative is $\partial_k \cdot v = d\delta_{kh}$, where δ_{kh} equals one for $k = h$ and zero otherwise; for an external momentum it vanishes. The integrated total derivative is zero in a region of convergence, and analytic continuation extends the relation to dimensional regularization [2, 1]. The same regularization prescription sets scaleless integrals to zero.

With a complete scalar-product basis, the scalar numerator $v \cdot \partial_k D_i$ can be written as $u_i + \sum_j v_{ij} D_j$. The coefficients u_i and v_{ij} depend on masses, invariants and the chosen momentum routing. Substituting in Eq. (7) gives

$$0 = \delta_v I_{\boldsymbol{\sigma}} - \sum_i \sigma_i u_i I_{\boldsymbol{\sigma} + \boldsymbol{e}_i} - \sum_{i,j} \sigma_i v_{ij} I_{\boldsymbol{\sigma} + \boldsymbol{e}_i - \boldsymbol{e}_j}, \tag{8}$$

Here $\delta_v = \partial_k \cdot v$, and the unit vector $\boldsymbol{e}_i$ increases index i by one. The derivative therefore relates the seed integral to integrals with shifted integer indices. Auxiliary denominators complete an otherwise incomplete scalar-product representation. When the denominator representation is dependent, additional algebraic relations are needed; the restricted vacuum family in Appendix D uses explicit mass partial fractions.

Suppose row q of such a system satisfies $\sum_j A_{qj} I_j = 0$ and $A_{qk} \neq 0$. Solving this row for the pivot integral gives

$$I_k = -\sum_{j \neq k} \frac{A_{qj}}{A_{qk}} I_j. \tag{9}$$

For a second identity row, or a target coefficient row r, subtract (A_{rk}/A_{qk}) times the zero identity. The resulting coefficients are

$$A'_{rj} = A_{rj} - \frac{A_{rk}}{A_{qk}} A_{qj}, \qquad A'_{rk} = 0. \tag{10}$$

Because the subtracted identity is zero, this operation preserves the target while removing its pivot entry. Choosing a different allowed source row can change the intermediate expressions and their cost without changing that equality. Repeated elimination removes the nonmaster columns and yields Eq. (3), provided each pivot coefficient is nonzero. Kinematic points at which a required denominator vanishes require a separate calculation.

B Reverse construction and the meaning of reference labels

The reverse construction provides targets with known answers because each checked source identity E_q is a linear combination of integrals that equals zero. Substituting these identities into Eq. (4) gives

$$T - \sum_m r_m M_m = \sum_q \lambda_q E_q = 0. \tag{11}$$

The equality fixes the master coefficients of the constructed expression. Expanding the source identities and collecting terms with equal index vectors then gives the target supplied to the model. Recording the operations, source rows, and seed–operator labels provides a path back to the masters. The coefficients λ_q and r_m may be rational functions, including constants as a special case. Algebraic mass partial-fraction identities are recorded separately because they have no IBP seed label.

Different zero identities can give different construction histories for the same target. A saved seed set is therefore sufficient, but need not be unique or smallest. Likewise, the saved row sequence need not use the fewest operations. A seed label records membership in one checked set, and a row label records the next step in one checked path. The final master coefficients and later steps do not enter the model inputs.

C Model specification and training objectives

This appendix specifies what each model learns and how its training objective is obtained. We begin with the seed network, whose symbolic target coefficients must first be represented by numerical inputs. We split each expression into tokens and count individual tokens and consecutive pairs and triples. A fixed signed map places these counts in 64 components. Different token groups can share a component, so equal representations do not prove equal expressions. Integral indices, complexity counts and term positions give further inputs. A shared hyperbolic-tangent map converts each term into a vector $\boldsymbol{h}_u \in \mathbb{R}^{32}$. Family symbols, master indices and candidate indices give separate 32-component vectors.

C.1 Seed representation and loss

The target representation includes the following summary of ordered term pairs:

$$\boldsymbol{h}_{\text{pair}} = \frac{1}{n_T(n_T-1)} \sum_{u \neq v} \psi_\theta([\boldsymbol{h}_u, \boldsymbol{h}_v, \boldsymbol{h}_u - \boldsymbol{h}_v, \boldsymbol{h}_u \odot \boldsymbol{h}_v]), \tag{12}$$

where n_T is the number of terms, square brackets concatenate vectors, $\odot$ multiplies components and ψ_θ returns 32 components. For $n_T = 1$, the summary is zero. This definition covers ordered pairs, not all larger term subsets. The candidate vector $\boldsymbol{u}_b$ weights target terms through scaled dot-product attention [9]:

$$w_{bu} = \frac{\exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_u/\sqrt{32})}{\sum_v \exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_v/\sqrt{32})}, \qquad \boldsymbol{v}_b = \sum_u w_{bu} \boldsymbol{h}_u. \tag{13}$$

The final input combines the mean and componentwise maximum of the term vectors with the pair, family, basis, seed, and attention-weighted term vectors. A hidden layer with 64 components produces g_θ in Eq. (5). Shared maps and aggregation follow Ref. [8], while stored positions retain the target term order. Reordering the candidate seeds changes only the order of the output scores.

If $y_b \in \{0, 1\}$ is recorded seed membership, a Bernoulli model assigns label likelihood $p_b^{y_b}(1 - p_b)^{1-y_b}$. Taking its negative logarithm gives $-y_b \log p_b - (1 - y_b) \log(1 - p_b)$. Averaging within

each candidate list and then across targets gives the implemented loss

$$\mathcal{L}_s = -\frac{1}{|\mathcal{D}_s|} \sum_{T \in \mathcal{D}_s} \frac{1}{|\mathcal{C}(T)|} \sum_{b \in \mathcal{C}(T)} [y_b \log p_b + (1 - y_b) \log(1 - p_b)]. \tag{14}$$

Here $\mathcal{D}_s$ is the training-target set. This objective treats each target equally and does not claim that true seed dependencies are statistically independent. Training uses AdamW [10] with learning rate 10^{-3}, weight decay 10^{-4}, gradient norm limit one and up to 32 targets per update. The threshold $1/2$ stays fixed during the expanded label test.

C.2 Row representation and loss

The row network compares the actions available at the current matrix state. Let $\mathcal{A}$ be that action list, and let $\boldsymbol{f}_a \in \mathbb{R}^{132}$ be the feature vector for action a. The encoder is $\boldsymbol{h}_a = \tanh(W\boldsymbol{f}_a + \boldsymbol{b})$ with 32 components. Concatenating $\boldsymbol{h}_a$ with the mean and componentwise maximum over $\mathcal{A}$ gives the input to a 64-component hidden layer. Its scalar output g_a defines

$$p(a \mid A, \mathcal{A}) = \frac{e^{g_a}}{\sum_{b \in \mathcal{A}} e^{g_b}}, \qquad \mathcal{L}_r = -\frac{1}{|\mathcal{D}_r|} \sum_{(A, \mathcal{A}, a^*) \in \mathcal{D}_r} \log p(a^* \mid A, \mathcal{A}). \tag{15}$$

Here A is the matrix, a^* is the recorded action, and $\mathcal{D}_r$ contains the training states with a choice. The softmax denominator normalizes the probabilities to one, and taking the negative logarithm of the recorded action's probability gives the loss. A state with one action has zero loss. Training uses the seed model's AdamW settings and 256 states per update. Trial updates modulo a prime provide cancellation features. The numerical size of a modular value does not measure the size of a real coefficient.

Later row models learn the cost of an action. One fit predicts the logarithm of the measured insertion time. Another counts the remaining forward insertions after that action, with Kira choosing later steps. If insertion j uses n_j source entries, its estimated cost is $1 + 2n_j$: one division, followed by a multiplication and subtraction for each entry. Thus the cost label is $C(S, q) = \sum_j (1 + 2n_j)$ for state S and choice q. The later tree fits

$$y_q = \frac{C(S, q) - C(S, 0)}{\max\{1, \max_b C(S, b) - \min_b C(S, b)\}}, \qquad \widehat{y}_q = h(\boldsymbol{x}_q - \boldsymbol{x}_0), \tag{16}$$

where 0 denotes Kira's native source and $\boldsymbol{x}_q$ contains eighteen current-row and overlap features. Weighted squared error gives each training family equal weight. Validation selects the tree and the required predicted improvement, and exact coefficients must agree before a measured continuation enters training. This count describes coefficient operations, while complete timings include the algebraic work inside each coefficient operation. Neither training objective proves a globally minimum path.

C.3 Equation features and tree fitting

The equation model uses a reference label derived from the support of an equation, meaning the set of integrals with nonzero coefficients. An equation label records whether this support appears in Kira's selected system. Table 14 groups the equation tree's seventeen inputs. They describe the seed and target before symbolic coefficient construction. For seed indices $\boldsymbol{\sigma}$ and target indices $\boldsymbol{a}$, the distance is $\sum_i |\sigma_i - a_i|$. The additional-power count is $\sum_i \max(\sigma_i - 1, 0)$, and the numerator rank is $\sum_i \max(-\sigma_i, 0)$. Neither quantity requires equation coefficients.

Training gives each physical profile equal total weight by assigning each row weight $1/(n_b B_g)$, where n_b is its batch size and B_g is the number of training batches in its profile. At a node, let

Table 14: Inputs to the original equation tree. The scattering seed trees omit the operator input and use sixteen features.

Group	Quantities	Number
Family and request	Loops, external momenta, index count, logarithm of target count	4
Seed	Positive indices, additional powers, numerator rank	3
Target limits	Excess seed powers and rank above target maxima	2
Sector	Whether requested; excess powers and rank above its target maxima	3
Target distance	Minimum index distance; whether the seed is a target	2
Operator	IBP operator index	1
Overall limits	Maximum target powers and numerator rank	2

w_+, w_- be the positive and negative total weights. Half the weighted binary Gini impurity is

$$H = \frac{w_+ w_-}{w_+ + w_-}. \tag{17}$$

Indeed, with $p = w_+/(w_+ + w_-)$, the weighted Gini expression is $(w_+ + w_-)[1 - p^2 - (1-p)^2] = 2H$. Each proposed split reduces H by the sum of its two children's values. The fit tests 25 feature quantiles between 0.02 and 0.98, rejects children below the stated population limit, and selects the largest positive reduction. This is the classification-tree construction of Ref. [12] with our bounded set of candidate splits.

A leaf with n_+ positive rows among n rows returns

$$p_{\text{leaf}} = \frac{n_+ + 1/2}{n + 1}. \tag{18}$$

Adding one half-count to each class gives a finite score even when a leaf contains only one class. The original tree keeps an equation when $p_{\text{leaf}} \geq 0.01$. These scores estimate reference-set membership; they are not calibrated probabilities that the whole reduction succeeds. The depth-eight and depth-twelve fits require at least 24 and eight rows in each child. The split procedure follows Ref. [12]. A basis check and fallback still test whether the selected system is sufficient.

C.4 Additional scattering seed and equation trees

The added scattering study fits a separate pair of trees for each of the three graph structures. The equation labels use the same selected-support rule as the original equation model. Let $y_{\sigma,o}$ be one when the native selected system contains the recorded equation support for seed σ and operator o. A seed receives label one when any of its operators has label one:

$$y_\sigma = \max_o y_{\sigma,o}. \tag{19}$$

For binary labels, this maximum is one exactly when at least one selected-support equation comes from the seed. This proves the stated membership rule. The label gives a useful reference set, not a proof that every labelled seed is necessary.

Each seed appears once per training batch. Its tree uses the sixteen features that do not depend on the operator. Thus every operator of that seed receives the same score. The early filter either permits or skips construction of all its equations. Kira still enumerates the seed region and applies its native symmetries. The equation tree uses all seventeen features and can keep different operators of the same seed.

Both fits have maximum depth twelve and at least eight records in each child. Each training batch has equal total weight. The split rule and leaf score are those derived above. Validation compares thresholds 0.001, 0.01 and 0.05 using full reduction time. Three repetitions contribute

to each validation median. The chosen threshold minimizes the geometric mean ratio over the rational and symbolic validation batches of its graph. Model parameters and thresholds are fixed before testing. For the nonplanar box, both thresholds are 0.05. For the pentagon, the seed and equation thresholds are 0.05 and 0.001. For the box with virtuality, they are 0.001 and 0.01. These fits create new small tree models; they do not change the original neural checkpoints. The reserved timing test gives no overall saving, so the application keeps native choices as its default.

D Exact reuse for the restricted vacuum family

The large cumulative saving in Section 6 follows from exact recurrences in a particular Euclidean vacuum family. We derive them here to state both the source of the saving and the restrictions on its use. The family has positive, distinct squared masses $\mu_j = \rho_j \mu$:

$$V_{\boldsymbol{b}}^{(L)} = \int \prod_{h=1}^{L} \frac{d^d \ell_h}{\pi^{d/2}} \frac{1}{\ell_1^2 \cdots \ell_L^2} \prod_{j=1}^{R} (Q^2 + \mu_j)^{-b_j}, \qquad Q = \sum_{h=1}^{L} \ell_h. \tag{20}$$

Here b_j is a nonnegative integer, $B = \sum_j b_j \geq 1$, R is the number of massive denominators, μ is a common squared-mass scale and ρ_j is a positive mass ratio. Each massless denominator has power one. All massive denominators share Q, so this family has special algebraic dependencies. Its prescribed masters are $M_j^{(L)} = V_{\boldsymbol{e}_j}^{(L)}$. The comparison uses $R = 2, 3, 4$ and $\mu = 1$ in the selected units.

D.1 Euler and mass partial-fraction identities

We first derive two relations that lower the massive propagator powers. Let f be the integrand without the measure. The simultaneous Euler derivative gives $\sum_h \ell_h \cdot \partial_{\ell_h} (\ell_i^2)^{-1} = -2(\ell_i^2)^{-1}$ for each massless factor. Also, $\sum_h \ell_h \cdot \partial_{\ell_h} Q^2 = 2Q^2$. Therefore

$$\sum_h \partial_{\ell_h} \cdot (\ell_h f) = \left[Ld - 2L - 2 \sum_j b_j \frac{Q^2}{Q^2 + \mu_j} \right] f. \tag{21}$$

Use $Q^2 = (Q^2 + \mu_j) - \mu_j$ and integrate the derivative as in Appendix A. This gives

$$0 = [L(d-2) - 2B] V_{\boldsymbol{b}}^{(L)} + 2 \sum_j b_j \mu_j V_{\boldsymbol{b}+\boldsymbol{e}_j}^{(L)}. \tag{22}$$

For two present massive denominators, the integrand identity $(Q^2 + \mu_i) - (Q^2 + \mu_j) = \mu_i - \mu_j$ separately gives

$$(\mu_i - \mu_j) V_{\boldsymbol{b}}^{(L)} = V_{\boldsymbol{b}-\boldsymbol{e}_i}^{(L)} - V_{\boldsymbol{b}-\boldsymbol{e}_j}^{(L)}. \tag{23}$$

This second relation is an algebraic partial fraction, so it introduces no additional IBP seed.

D.2 A recurrence shared across loop orders

To make the common loop dependence explicit, define $z = L(d-2)$ and $W_{\boldsymbol{b}}^{(L)} = \mu^{B-1} V_{\boldsymbol{b}}^{(L)}$. Multiplication by this power of μ gives W the same mass dimension as the masters. Substituting in Eqs. (22) and (23) removes the common scale. If $\boldsymbol{C}_{\boldsymbol{b}}(z)$ is the vector of master coefficients of $W_{\boldsymbol{b}}^{(L)}$, then

$$\boldsymbol{C}_{\boldsymbol{b}}(z) = \frac{\boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_i}(z) - \boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_j}(z)}{\rho_i - \rho_j}, \qquad b_i, b_j > 0, \tag{24}$$

$$\boldsymbol{C}_{k\boldsymbol{e}_j}(z) = \frac{2(k-1) - z}{2(k-1)\rho_j} \boldsymbol{C}_{(k-1)\boldsymbol{e}_j}(z), \qquad k \geq 2. \tag{25}$$

The initial vector is $\boldsymbol{C}_{\boldsymbol{e}_j} = \boldsymbol{e}_j$. Every recurrence lowers B, so it terminates at the prescribed masters. Its coefficients depend on L only through z. Induction on B shows that they are polynomials in z with rational coefficients at fixed rational ρ_j. One calculation can therefore supply all requested loop orders by substituting $z = L(d-2)$.

D.3 An independent coefficient check

The recurrence result is checked by obtaining the mixed-denominator coefficients through a Taylor partial-fraction expansion. For a dimensionless auxiliary variable x, write

$$\prod_i (x+\rho_i)^{-b_i} = \sum_{j:b_j>0} \sum_{k=1}^{b_j} \frac{a_{jk}}{(x+\rho_j)^k}. \tag{26}$$

At the pole $x = -\rho_j$, set $t = x + \rho_j$. Multiplying by t^{b_j} and expanding the regular factors shows that the principal-part coefficient is

$$a_{jk} = [t^{b_j-k}] \prod_{i\neq j} (\rho_i - \rho_j + t)^{-b_i}. \tag{27}$$

Here $[t^n]$ extracts Taylor coefficient n and factors with $b_i = 0$ are one. Applying Eq. (25) repeatedly to each pole term gives

$$C_{\boldsymbol{b}j}(z) = \sum_{k=1}^{b_j} a_{jk} \prod_{h=1}^{k-1} \frac{2h-z}{2h\rho_j}, \qquad V_{\boldsymbol{b}}^{(L)} = \mu^{1-B} \sum_j C_{\boldsymbol{b}j}(L(d-2)) M_j^{(L)}. \tag{28}$$

An empty product is defined as one. This Taylor calculation checks every returned polynomial coefficient before the substitution for z and does not follow the chosen sequence of pair recurrences. The derivation requires distinct ρ_j, unit powers for the massless propagators, and the shared momentum Q. These assumptions delimit the family to which the result applies.

E Measurement definitions and comparison conditions

The comparisons use separate measures for label prediction, algebraic completion, and elapsed time. For the label measures, let TP be the number of selected reference seeds, FP the number of selected seeds absent from the reference set, and FN the number of omitted reference seeds. Precision is $P = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ and recall is $R = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. Defining F_1 as their harmonic mean gives

$$F_1 = \frac{2}{P^{-1}+R^{-1}} = \frac{2PR}{P+R} = \frac{2\mathrm{TP}}{2\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}. \tag{29}$$

Complete-set agreement counts targets for which the entire predicted seed set equals the recorded set. Algebraic sufficiency instead counts targets that reach the prescribed master basis using the selected equations. Since a target can have several sufficient seed sets, these measures need not agree with each other or with the individual-label scores in Figure 1.

For timing, let T_{abr} be the measured time for method a, batch b and repetition r. We use $\widetilde{T}_{ab} = \mathrm{median}_{r=1,\dots,5} T_{abr}$. For the later native comparisons, if physical group g contains batches B_g, define the equal-group mean logarithmic ratio

$$\bar{\ell} = \frac{1}{G} \sum_{g=1}^{G} \frac{1}{|B_g|} \sum_{b\in B_g} \log \frac{\widetilde{T}_{ab}}{\widetilde{T}_{cb}}, \qquad R_{a/c} = e^{\bar{\ell}}. \tag{30}$$

Here c identifies the control and G is the number of physical groups. Exponentiating the mean log ratio gives the geometric mean; ratios below one indicate lower time. Reversing numerator and

denominator changes $\bar{\ell}$ to $-\bar{\ell}$, so speedup is $S = 1/R_{a/c}$ and the time reduction is $100(1-R_{a/c})\%$. The cumulative study uses one batch per group. The supplied-equation seed comparison instead gives each complete paired batch equal weight; its interval resamples six related kinematic groups. Following bootstrap resampling [14], reported descriptive 95% intervals resample complete groups 10,000 times while retaining paired methods and repetitions. They describe case-to-case variation in these experiments. We do not infer new uncertainty intervals for figures whose source summaries provide only point estimates.

The supplied-equation seed test includes target preparation, inference, early basis checking, reduction, result checking and fallback. It excludes original IBP-pool generation. It rotates methods within each family and target batch on the same node, and includes only complete paired batches in its ratio summary while preserving incomplete and skipped trials in the accounting.

The native equation and transfer tests include input preparation, model loading and evaluation, Kira startup, native IBP generation, elimination, basis and coefficient checks, cleanup and every fallback. Independent reference discovery and the later independent check lie outside the timer. In the equation test, the native and learned arms receive the same executable-level improvements. The eight reserved profiles test new masses and invariants within known structures; the seven transfer cases additionally test new structures and symbolic invariants.

The cumulative test includes fresh-worker startup, input loading, supplied Euler and mass-identity construction, algebra, verification, output saving and complete output reading. Eight groups use maximum massive powers 8, 10, 12 or 14, with two groups at each bound. They request 16, 32, 48 or 64 targets per profile, capped by availability. Each group contains three mass profiles and all loop orders 1–12. Paired methods use the same resources, and their order changes between repetition blocks. Each method recomputes the reduction. Waiting for resources, offline training and the later independent check remain outside all reported production times.

The initial live-matrix comparison uses three repetitions of four methods on seven physical cases, giving 84 workflows. Its complete ratio is the geometric mean of the seven paired median ratios and carries no reported uncertainty interval. The later native implementation comparison uses five repetitions and sixteen batches; it repeats earlier physical profiles to measure the implementation change.

The reported numbers are measurements from this study. We keep the original label tests, seed tests, row tests, equation tests, scattering tests and vacuum benchmark separate. This prevents differences in targets or timed work from being mistaken for a combined model gain. The archived notes preserve earlier experiments, including negative results.

F Integral families and kinematic conventions

The physical scope of each test is determined by its denominator momenta, masses, and external scalar products. This appendix gives those definitions for the training and test families. Together with an index vector, or a rational linear combination of index vectors in one family, they specify the targets without requiring an individual listing of the full collection. Unless stated otherwise, $D_i = q_i^2 - m_i^2 + i0$ and the measure is that of Eq. (1). The Euclidean vacuum family keeps the sign convention in Eq. (20).

F.1 Initial one-loop and two-loop families

For the one-loop tadpole, $q_1 = \ell$ and $m_1^2 = \mu > 0$. For the bubbles,

$$\boldsymbol{q} = (\ell, \ell - p), \qquad p^2 = s, \qquad (m_1^2, m_2^2) \in \{(0,0), (\mu,0), (\mu,\mu), (\mu_1,\mu_2)\}. \tag{31}$$

The symbols μ, μ_1, μ_2 denote squared masses, with $\mu_1 \neq \mu_2$ in the unequal-mass case. The triangle routing is

$$\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2), \qquad (p_1^2, p_2^2, (p_1 + p_2)^2) = (u_1, u_2, u_3). \tag{32}$$

Expanding $(p_1 + p_2)^2$ gives $2p_1 \cdot p_2 = u_3 - u_1 - u_2$. Three massless triangle classes use virtualities $(0, 0, s)$, $(0, u_2, u_3)$ and (u_1, u_2, u_3). Two equal-mass classes, with all internal squared masses μ, use $(0, 0, s)$ and (u_1, u_2, u_3). Each displayed nonzero virtuality is independent at generic kinematics.

The initial massless boxes use four incoming external momenta, $p_4 = -p_1 - p_2 - p_3$, and

$$\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3), \quad s = (p_1 + p_2)^2, \quad t = (p_2 + p_3)^2. \tag{33}$$

Writing $u_i = p_i^2$, the three classes have $(u_1, u_2, u_3, u_4) = (0, 0, 0, 0)$, $(0, 0, 0, u_4)$ and (u_1, u_2, u_3, u_4). Expanding the definitions of s, t and p_4^2 gives

$$2p_1 \cdot p_2 = s - u_1 - u_2, \quad 2p_2 \cdot p_3 = t - u_2 - u_3, \quad 2p_1 \cdot p_3 = u_4 + u_2 - s - t. \tag{34}$$

These definitions give thirteen one-loop classes. The initial two-loop massless two-point family uses

$$\boldsymbol{q} = (\ell_1, \ell_2, \ell_1 - \ell_2, \ell_1 - p, \ell_2 - p), \qquad p^2 = s, \qquad m_i^2 = 0. \tag{35}$$

The equal-mass sunrise uses $\boldsymbol{q} = (\ell_1, \ell_2, p - \ell_1 - \ell_2, p - \ell_1, p - \ell_2)$, with masses $(\mu, \mu, \mu, 0, 0)$ and $p^2 = s$. Its last two entries are auxiliary: $a_4, a_5 \leq 0$. The equal-mass vacuum sunset has $\boldsymbol{q} = (\ell_1, \ell_2, \ell_1 - \ell_2)$ and all three squared masses μ. These three classes complete the initial sixteen. Their subsectors include the integrals obtained by removing physical denominators.

F.2 Additional one-loop mass and invariant assignments

The 984 added box families use a more general assignment than the three initial massless boxes:

$$D_i = (\ell - Q_i)^2 - \mu_{\alpha_i} + i0, \quad Q_1 = 0, \quad \mu_0 = 0, \quad \alpha_i \in \{0, 1, 2, 3\}. \tag{36}$$

The nonzero squared masses μ_1, μ_2, μ_3 are independent. Repeated labels impose equal masses. The three external shifts Q_2, Q_3, Q_4 have squared differences

$$\Delta_{ij} = (Q_i - Q_j)^2 = y_{\beta_{ij}}, \quad y_0 = 0, \quad \beta_{ij} \in \{0, 1, 2, 3\}. \tag{37}$$

The nonzero y_1, y_2, y_3 are independent invariants. Expanding a squared difference gives the Gram matrix

$$Q_i \cdot Q_j = \frac{\Delta_{1i} + \Delta_{1j} - \Delta_{ij}}{2}, \qquad i, j \in \{2, 3, 4\}. \tag{38}$$

The construction rejects singular sampled Gram matrices and identifies assignments related by propagator permutations or by relabelling nonzero masses and invariants. A fixed order selects 984 of 1,673 admissible inequivalent profiles after excluding the original massless boxes. This extends the initial collection to 1,000 families with 9,678 family-local masters and 240,472 targets before the higher-loop additions. Each profile gives a symbolic family through its zeros and equality conditions. The single-target seed timing comparison uses twelve reserved instances of these boxes. Its targets follow the reverse construction in Appendix B, rather than the native target enumeration below.

F.3 Native multiloop comparisons

The sunrise comparisons allow independent masses on the same three physical propagators. The other native comparison families use

$$\begin{aligned}\boldsymbol{q}_{\mathrm{DB}} &= (\ell_1, \ell_1 - p_1, \ell_1 - p_1 - p_2, \ell_2, \ell_2 - p_1 - p_2, \ell_2 - p_1 - p_2 - p_3,\\ &\qquad \ell_1 - \ell_2, \ell_1 - p_3, \ell_2 - p_1), \qquad (39)\\ \boldsymbol{q}_{\mathrm{V}} &= (\ell_1, \ell_2, \ell_3, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3), \qquad (40)\\ \boldsymbol{q}_{\mathrm{L}} &= (\ell_1, \ell_1 - p, \ell_2, \ell_2 - p, \ell_3, \ell_3 - p, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3). \qquad (41)\end{aligned}$$

The double box has two loops and seven physical denominators; its last two entries are auxiliary. The vacuum has three loops and six physical denominators. The ladder has three loops and eight physical denominators; its last entry is auxiliary. Auxiliary indices remain nonpositive. The massless double box has $p_i^2 = 0$ and uses $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = t$, $2p_2 \cdot p_3 = -s - t$. This local definition of t differs from the initial box convention because the routing differs.

For the final early-equation test, the sunrise squared-mass vectors are $(2, 7, 11)$ and $(3, 7, 13)$, with $p^2 = -101, -103$. The double-box profiles are $(s, t) = (-101, -103), (-107, -109)$. The vacuum squared-mass vectors are $(1, 1, 1, 1, 1, 11)$ and $(1, 1, 1, 13, 13, 13)$. The ladder has squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ and $p^2 = -101, -103$. All values use a common squared-momentum unit. These choices keep d symbolic and give coefficients in $\mathbb{Q}(d)$.

The target order starts with the full-sector corner, then adds each single extra propagator power and each available single auxiliary numerator. The construction repeats after removing one, two and three physical denominators, retains at least two, and removes duplicates. The comparison takes 21 sunrise targets and batches of 32 and 128 targets for each other profile. This gives the fourteen batches in Section 4.3.

F.4 New scattering structures and symbolic invariants

The massless nonplanar box uses the routing of Ref. [4]:

$$\begin{aligned}\boldsymbol{q}_{\mathrm{NP}} = (&\ell_1, \ell_2, p_1 - \ell_1, p_2 - \ell_2, p_1 + p_2 - p_3 - \ell_1,\\ &p_1 - \ell_1 + \ell_2, p_1 - p_3 - \ell_1 + \ell_2, \ell_1 - p_3, \ell_2 - p_1 - p_2). \qquad (42)\end{aligned}$$

The first seven entries are physical propagators, and the final two are auxiliary entries. Here $p_1^2 = p_2^2 = p_3^2 = (p_1 + p_2 - p_3)^2 = 0$, $s = (p_1 + p_2)^2$ and $t = (p_1 - p_3)^2$. Expanding these squares gives $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = -t$ and $2p_2 \cdot p_3 = s + t$.

The massless pentagon uses $\sum_{i=1}^{5} p_i = 0$, $p_i^2 = 0$ and

$$\boldsymbol{q}_{\mathrm{P}} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3, \ell - p_1 - p_2 - p_3 - p_4). \qquad (43)$$

All five denominators are physical. The independent invariants are $(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$, where $s_{ij} = (p_i + p_j)^2$. Squaring $p_1 + p_2 + p_3 = -(p_4 + p_5)$ gives $2p_1 \cdot p_3 = s_{45} - s_{12} - s_{23}$; the other nonadjacent products follow by cyclic relabelling. The box with an external virtuality uses $\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_3)$, $p_1^2 = p_2^2 = p_3^2 = 0$, and $v = (p_1 + p_2 - p_3)^2$. With the nonplanar convention for s, t, expanding v gives $2p_2 \cdot p_3 = s + t - v$.

The rational nonplanar profiles are $(s, t) = (-149, -147), (-167, -157)$. The pentagon profiles are $(-51, -53, -57, -59, -63)$ and $(-71, -73, -77, -79, -83)$ in the displayed invariant order. Each rational profile supplies sixteen targets. The symbolic nonplanar, pentagon and virtuality-box tests supply ten, six and eight targets. They start with the corner and single additional powers, followed by auxiliary numerators and single removed denominators where available. These definitions give the 88 targets of the transfer test.

F.5 Parameter points and targets for the additional training

The new scattering study uses the same three denominator lists and scalar-product conventions defined above. For integer profile number n, we choose

$$(s,t)_{\mathrm{NP}} = (-113-18n, -127-10n), \tag{44}$$

$$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\mathrm{P}} = (-11-20n, -13-20n, -17-20n, -19-20n, -23-20n), \tag{45}$$

$$(s,t,v)_{\mathrm{box}} = (-113-18n, -127-10n, -149-12n). \tag{46}$$

These are specified input values in a common squared-momentum unit. Training uses $n = 10, 11$, validation uses $n = 12$, and testing uses $n = 13, 14$. The dimension d remains symbolic in every case. Each symbolic batch instead keeps all of its displayed invariants independent.

The target list starts with the full-sector corner. It then raises each physical denominator power once and adds each single auxiliary numerator. The same sequence follows for each possible single removed physical denominator. Rational batches take the first sixteen entries. Symbolic training takes entries 0–3, validation takes 4–7, and testing takes 8–11, with indices starting at zero. We verify that the three symbolic lists are disjoint for each structure. Rational batches reuse index templates at different parameter points, so this split does not claim entirely unseen target patterns in every representation.

G Master bases and dimensionally consistent target construction

This appendix explains the checks used to identify the stored master bases and to combine target terms with consistent mass dimensions. We first count the scalar products that an independent denominator representation must cover:

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LE. \tag{47}$$

There are L loop squares and $L(L-1)/2$ distinct loop cross-products, followed by LE products with the E independent external momenta. This count supplies a direct rank check for the independent denominator lists. The restricted vacuum parent in Eq. (20) instead uses the explicitly dependent massive denominators and their partial fractions.

For index vector $\boldsymbol{a}$, define the positive-index sum $r = \sum_i \max(a_i, 0)$, numerator rank $s_- = \sum_i \max(-a_i, 0)$ and additional-power count $b = \sum_i \max(a_i - 1, 0)$. If the top sector has t_{top} physical denominators, the initial Kira basis calculation uses seed limits $(r, s_-, b) \le (t_{\mathrm{top}}+2, 2, 2)$ and probe limits $(t_{\mathrm{top}}+1, 1, 1)$, component by component. Both include all subsectors. The basis check then increases every limit by one and repeats at a second finite-field point. The original sixteen definitions give the same master labels in all three calculations, for 48 completed basis checks. Such agreement supports stability in the tested domains; it does not prove completeness for all integer indices [4, 5].

The initial sixteen classes contain 49 master entries when each family is counted separately. The tadpole contributes one. The four bubble classes give one, two, two and three, in their stated order. The five triangle classes give one, two, four, three and five. The three box classes give three, four and eleven. The massless two-loop two-point, sunrise and vacuum sunset classes give two, three and two. For example, the equal-mass sunrise basis is

$$M_1 = I_{(1,1,0,0,0)}, \qquad M_2 = I_{(1,1,1,0,0)}, \qquad M_3 = I_{(1,1,1,-2,0)}. \tag{48}$$

The first integral separates into two tadpoles, and the last contains the numerator $[(p-\ell_1)^2]^2$. These integral labels define the basis even when its numerical values have not been evaluated. Reduction training uses the relations to that basis, so it does not require those values. Each later family instance likewise retains its explicit master indices and parameter assignment.

The mass dimension of Eq. (1) is $Ld-2\sum_i a_i$: each loop measure contributes d powers of mass and each inverse propagator contributes two. In the restricted vacuum data, $[V_{\boldsymbol{b}}^{(L)}] = Ld-2L-2B$. Thus targets of different total powers can be combined consistently as

$$T = \sum_{\nu} \kappa_\nu d^{\eta_\nu} \mu^{B_\nu - B_*} V_{\boldsymbol{b}_\nu}^{(L)}, \qquad B_* = \max_\nu B_\nu. \tag{49}$$

Every term has dimension $Ld-2L-2B_*$. The construction uses $\eta_\nu \in \{0,1\}$ and $\kappa_\nu = n_\nu/q_\nu$, with $n_\nu \in \{-3,-2,-1,1,2,3\}$ and $q_\nu \in \{1,2,3\}$. Each expression contains one, two or three distinct roots. Individual roots have $2 \le B \le 5$; Euler seeds have $1 \le B \le 5$, and mass identities extend through $B = 6$. Each added family contains 128 distinct targets, including individual roots. The identity pools contain 35, 160 or 545 rows for two, three or four massive denominators.

For integer profile number n, the generated mass ratios are $R = 2 + ((n-1) \bmod 3)$, $\rho_1 = 1$ and $\rho_{j+2} = 2 + 3n + j(n+2)$ for $0 \le j \le R-2$. Profiles $n = 1, \ldots, 100$ enter at loop orders 2–12; profiles 101–1350 enter at orders 1–12. Distinct masses make the partial-fraction denominators nonzero. A profile remains in the same training or testing group at every loop order. These rules state the extent of the high-loop collection and its restricted algebraic structure.

G.1 Analytic check of the restricted masters

The recurrences in Appendix D reduce the specified family to $M_j^{(L)}$. We can also derive their values to check this choice. Write $\delta = d/2$, $\alpha = L(\delta-1)$ and $c = L-(L-1)\delta$. The symbol Γ denotes the Euler gamma function, and a, b below are denominator powers. The standard Euclidean two-denominator convolution follows from Feynman parameters [1]:

$$\begin{aligned} J(a,b;Q) &= \int \frac{d^dk}{\pi^\delta} \frac{1}{(k^2)^a((Q-k)^2)^b} \\ &= (Q^2)^{\delta-a-b} \frac{\Gamma(a+b-\delta)\Gamma(\delta-a)\Gamma(\delta-b)}{\Gamma(a)\Gamma(b)\Gamma(2\delta-a-b)}. \end{aligned} \tag{50}$$

To obtain this expression, combine the denominators with

$$\frac{1}{A^a B^b} = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^1 \frac{x^{a-1}(1-x)^{b-1}\,dx}{[xA+(1-x)B]^{a+b}}. \tag{51}$$

Shifting k completes the square and gives the radial denominator $k^2 + x(1-x)Q^2$. The radial integral equals $\Gamma(a+b-\delta)[x(1-x)Q^2]^{\delta-a-b}/\Gamma(a+b)$. The remaining beta integral is $\Gamma(\delta-b)\Gamma(\delta-a)/\Gamma(2\delta-a-b)$, which yields Eq. (50).

At fixed Q, successive convolution of the L unit-power massless factors gives

$$C_L (Q^2)^{-c}, \qquad C_L = \frac{\Gamma(c)\Gamma(\delta-1)^L}{\Gamma(\alpha)}. \tag{52}$$

For $L = 1$ the coefficient is one. Applying Eq. (50) with $a = c$, $b = 1$ advances c to $c+1-\delta$ and proves the formula by induction. The final integral over Q then gives

$$\begin{aligned} M_j^{(L)} &= \frac{C_L}{\Gamma(\delta)} \int_0^\infty \frac{x^{\alpha-1}\,dx}{x+\mu_j} \\ &= \mu_j^{\alpha-1} \frac{\Gamma(c)\Gamma(\delta-1)^L\Gamma(1-\alpha)}{\Gamma(\delta)}. \end{aligned} \tag{53}$$

The change $x = \mu_j y$ reduces the integral to $\Gamma(\alpha)\Gamma(1-\alpha)$ for $0 < \operatorname{Re}\alpha < 1$. Analytic continuation gives the dimensionally regulated result [1]. Distinct positive fixed mass ratios give distinct exponentials $\rho_j^{L(d/2-1)-1}$ as functions of d. After common factors cancel, a rational-function

relation would have a unique largest exponential as real d increases. Rational coefficients grow at most as powers and cannot cancel that term. Removing its zero coefficient and repeating establishes independence over $\mathbb{Q}(d)$ at these fixed mass ratios. Special dimensions and coincident masses require their own analysis.

H Finite minimum-operation checks

The saved construction supplies a valid reduction path, but a minimum-cost claim requires a defined set of alternatives. We therefore specify the allowed equations, row operations, and cost before checking all paths for two small systems. These bounded examples make precise the distinction between a known path and a certified minimum. Define the massless bubble $B_{a,b} = I_{(a,b)}$ with denominators $\ell^2 + i0$ and $(\ell - p)^2 + i0$, where $p^2 = s \neq 0$. Contracting the derivative with ℓ uses $2\ell \cdot (\ell - p) = \ell^2 + (\ell - p)^2 - s$. Equation (7) then gives

$$0 = (d - 2a - b)B_{a,b} - bB_{a-1,b+1} + bsB_{a,b+1}. \tag{54}$$

Changing variables $\ell \mapsto p - \ell$ exchanges a, b and gives the second operator identity. Integrals with only a massless denominator vanish in dimensional regularization [1]. The two operators at $(1, 1)$, the first at $(2, 1)$ and the second at $(1, 2)$ therefore give

$$\begin{pmatrix} 0 & s & 0 & d-3 \\ s & 0 & 0 & d-3 \\ d-5 & -1 & s & 0 \\ -1 & d-5 & s & 0 \end{pmatrix} \begin{pmatrix} B_{2,1} \\ B_{1,2} \\ B_{2,2} \\ B_{1,1} \end{pmatrix} = 0. \tag{55}$$

The first two rows yield $B_{2,1} = B_{1,2} = (3 - d)B_{1,1}/s$. Substituting in the third yields $B_{2,2} = (d - 6)(d - 3)B_{1,1}/s^2$.

For the massive tadpole $T_a = \int d^d\ell\, (\ell^2 - m^2 + i0)^{-a}/(i\pi^{d/2})$, the same derivative gives

$$0 = dT_a - 2a \int \frac{d^d\ell}{i\pi^{d/2}} \frac{\ell^2}{(\ell^2 - m^2 + i0)^{a+1}} = (d - 2a)T_a - 2am^2 T_{a+1}. \tag{56}$$

Seeds $a = 1, 2, 3$ reduce T_2, T_3, T_4 to T_1. We keep the master column protected in both systems. An allowed step selects a nonzero pivot in an unprocessed nonmaster column, normalizes its row and eliminates that column from the other rows. The cost counts one division per nonzero nonpivot source entry when normalization is needed, plus one multiplication and one subtraction for each nonpivot source-entry update. It excludes assignments and the internal polynomial operations of a rational-function operation.

Let S be the exact matrix state, $\mathcal{A}(S)$ its allowed actions, $c(S, a)$ the stated step cost and S_a the resulting state. Splitting every complete path into its first action and its remaining path gives the finite recurrence [15]

$$V(S) = \min_{a \in \mathcal{A}(S)} \{c(S, a) + V(S_a)\}, \qquad V(S_{\text{done}}) = 0. \tag{57}$$

Each action removes an unprocessed column. Induction on the number of remaining columns proves the minimum after all actions have been evaluated. Repeating this calculation for every source-row subset of nonmaster rank three also minimizes over initial equation selection.

Our bubble check covers 113 states, 234 transitions and 152 complete paths across five sufficient subsets. Its minimum is seven coefficient operations with subset selection and thirteen with the full pool. The tadpole check covers 24 states, 32 transitions and fifteen paths; its minimum is seven operations with its single sufficient subset. Each pool requires at least three equations. The smallest number of distinct seed labels is two for the bubble and three for the tadpole. Exact replay and six native Kira master-coefficient comparisons agree. These are minima for the displayed identities, protected basis, allowed operations and cost. They do not establish minimum elapsed time or an optimum over all IBP identities.

I Learning the cost of a complete reduction

This appendix defines the complete-time prediction problem, derives the fitted models, and specifies the physical inputs used in Section 5.1. The cost model receives a feature vector $\boldsymbol{f}_c$ for target batch c. Its entries are the number of loops, external momenta, inverse propagators, physical propagators and symbolic invariants; $\log(1 + n_c)$, where n_c is the number of targets; the maximum dot count, maximum numerator rank, mean dot count and mean numerator rank; the number of target sectors; and seven indicators for the graph structures. An indicator is one for its graph and zero for the other graphs. Together these quantities give eighteen inputs, all available from the requested targets and family before reduction.

For an index vector $\boldsymbol{a}$, define

$$r(\boldsymbol{a}) = \sum_i \max(a_i, 0), \qquad s(\boldsymbol{a}) = \sum_i \max(-a_i, 0), \qquad q(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0). \tag{58}$$

Here r is the positive index sum, s is the numerator rank, and q is the number of extra denominator powers, called dots. These definitions follow directly by counting the denominator and numerator factors. The initial bounds $B_c = (r_c, s_c, q_c)$ are the componentwise maxima over the targets. The trial seed regions use B_c, $B_c - (1, 1, 1)$, $B_c - (1, 0, 1)$, or $B_c + (1, 1, 1)$. Negative bounds are set to zero. The choices change the region in which equations are generated. Since a smaller region can omit required relations, every attempt must still pass the prescribed-basis check.

Let t_{cj} be the median complete time of strategy j on training batch c, and let t_{c0} be the time of the integrated native strategy. The training response is

$$y_{cj} = \log \frac{t_{cj}}{t_{c0}}. \tag{59}$$

Each time includes unsuccessful attempts and the eventual successful fallback, so a strategy that omits necessary equations is penalized by the extra calculation. The logarithm preserves the ordering of positive time ratios; minimizing the response therefore also minimizes the complete time relative to the common baseline.

We fit constant, tree, linear and nearest-neighbour predictors of this response. The constant predictor averages each strategy's training responses. A tree splits the feature space and uses a constant vector within each leaf, following regression-tree construction [12]. For a leaf containing training set A, minimizing $\sum_{c\in A}(y_{cj} - v_j)^2$ gives

$$\frac{\partial}{\partial v_j} \sum_{c\in A} (y_{cj} - v_j)^2 = 2|A| v_j - 2 \sum_{c\in A} y_{cj} = 0, \qquad v_j = \frac{1}{|A|} \sum_{c\in A} y_{cj}. \tag{60}$$

Each split minimizes the sum of squared errors over the seven strategies. The tested maximum depths are two, four and six. Each depth is tested with a minimum of either one or three training batches per child.

For the linear models, we first subtract the training mean of each feature and divide by its training population standard deviation, computed with divisor equal to the number of training batches. A constant feature uses divisor one. Let Z contain these standardized feature rows with a leading intercept column, and let Y contain the responses. Ridge regression minimizes

$$J(W) = \|ZW - Y\|_F^2 + \lambda \operatorname{tr}(W^T P W), \qquad P = \operatorname{diag}(0, 1, \ldots, 1). \tag{61}$$

Here W is the matrix of fitted coefficients, $\|\cdot\|_F^2$ is the sum of squared matrix entries, and P leaves the intercept unpenalized. Differentiating each entry of W gives

$$\nabla_W J = 2Z^T(ZW - Y) + 2\lambda P W = 0, \qquad (Z^T Z + \lambda P)W = Z^T Y. \tag{62}$$

We solve this system for $\lambda = 0.1, 1, 10$.

The nearest-neighbour models estimate a cost from similar training batches. Eligible neighbours must have the same graph, loop count, external-momentum count, propagator counts, and number of symbolic invariants. They choose up to two or four training batches with the smallest squared distance in standardized features. The predicted response is the mean of their response vectors. The leaf-mean derivative above also derives this local constant estimate. If no eligible neighbour exists, the predictor returns equal costs and the selector uses native choices.

Let j_* be the strategy with the smallest predicted logarithmic cost $\widehat{y}_j(\boldsymbol{f})$. Validation selects a margin $\gamma \in \{0, 0.05, 0.15\}$, and the model chooses the strategy as follows:

$$\widehat{j}(\boldsymbol{f}) = \begin{cases} j_*, & \widehat{y}_{j_*}(\boldsymbol{f}) < \widehat{y}_0(\boldsymbol{f}) - \gamma, \\ 0, & \text{otherwise.} \end{cases} \tag{63}$$

Subtracting the two predicted logarithmic costs shows that the condition requires a predicted relative cost below $e^{-\gamma}$. Validation selects the model and margin with the smallest geometric mean time ratio. The selected settings remain fixed during reserved tests. This finite comparison establishes the best measured candidate within the study. It does not prove a globally optimal reduction strategy.

I.1 Parameter points and target lists

The comparison uses the sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality, with the same denominator routings and kinematic conventions as the preceding tests. The definitions below specify the parameter values and target lists for each split. Let h identify a rational parameter profile. Training uses $h = 41, 42$, validation uses $h = 43$, and testing uses $h = 44, 45$. In a common squared-momentum unit, the sunrise has squared masses $(2, 7, 11)$ and $p^2 = -1009 - 12h$. The planar box has $(s, t) = (-1009 - 12h, -1021 - 14h)$. The three-loop vacuum has squared masses $(1, 1, 1, 1, 1, 53 + 2h)$. The ladder keeps squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ and has $p^2 = -1009 - 12h$. The scattering assignments are

$$(s, t)_{\mathrm{NP}} = (-113 - 18h, -127 - 10h), \tag{64}$$

$$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\mathrm{P}} = (-11 - 20h, -13 - 20h, -17 - 20h, -19 - 20h, -23 - 20h), \tag{65}$$

$$(s, t, v)_{\mathrm{box}} = (-113 - 18h, -127 - 10h, -149 - 12h). \tag{66}$$

These assignments fix the rational parameter values in each profile while leaving the dimension symbolic. The additional symbolic batches instead retain all independent invariants of their scattering graph as symbols.

For the four earlier multiloop graphs, we retain the ordered target list that starts with the corner, raises each physical denominator power once, and then introduces each single auxiliary numerator. We repeat these steps after removing one, two and three physical denominators, retain at least two, and remove duplicates. Each rational profile supplies a four-target batch and a batch of the first 32 targets, or all 21 targets for the sunrise. The three scattering graphs use the same construction with at most one removed denominator. Their larger batches have 32, 31 and 21 targets for the nonplanar box, pentagon and virtuality box.

For the symbolic nonplanar box and pentagon, training takes target entries 12–15, validation takes 16–19, and testing takes 20–23, with numbering starting at zero. These lists do not overlap. For the symbolic virtuality box, the respective single targets are $(3, 1, 1, 1)$, $(1, 3, 1, 1)$ and $(1, 1, 3, 1)$. These targets have two extra powers on one physical denominator. The rational batches reuse target patterns at different parameter points.

The joint-request test uses the larger rational batch at $h = 44$ for each graph and the three symbolic test batches. The symbolic virtuality box also includes $(1, 1, 1, 3)$, so it can supply two

disjoint requests. A group of n targets is divided after entry $\lfloor n/2 \rfloor$, with the first request containing that many entries. The two requests share the same family definition and are available together, and their union contains each target once. By linearity, selecting the corresponding target rows from the joint reduction returns the coefficients for each original request. This separation leaves every coefficient unchanged and requires no new integral identity. The test reuses reserved parameter points and keeps the learned model fixed.

I.2 Combining the measured times

Let m_{cj} be the median of the five test times for batch c and method j, and let method zero be its stated native control. For graph g with batch set C_g, we define

$$R_{gj} = \exp\left[\frac{1}{|C_g|}\sum_{c\in C_g}\log\frac{m_{cj}}{m_{c0}}\right], \qquad R_j = \exp\left[\frac{1}{7}\sum_{g=1}^{7}\log R_{gj}\right]. \tag{67}$$

The logarithm of a product equals the sum of the logarithms, so each expression is the corresponding geometric mean of the time ratios. The second expression assigns equal weight to each graph even when the numbers of batches differ. A ratio $R_j < 1$ means less time. The percentage saving is $100(1 - R_j)$, and the reciprocal $1/R_j$ is the speedup.

For the descriptive intervals, we sample seven graph labels with replacement, calculate their mean logarithmic ratio, and exponentiate. We repeat this 10,000 times and report the 2.5th and 97.5th percentiles. This procedure describes variation among the seven tested graphs. It does not provide a population guarantee for other integral families. The joint-request comparison uses the same calculation, with separate native requests as its first control. Its parameter points overlap the main reserved test, so it is an additional comparison on the same test population.

J Separate cost models for each graph

The graph-specific study changes the target batches and available strategies while retaining the seven denominator definitions and parameter formulas in Appendix I. This appendix specifies those changes and the rule for deciding whether a new predictor is used. Development uses profile $h = 61$, training uses $h = 62, 63$, validation uses $h = 64$, and the reserved test uses $h = 65, 66$. The dimension remains symbolic. Related target batches share the same split, so their rational kinematic values cannot pass from the training set into the reserved set. The graph structures and target patterns occur in both sets.

Let P be the number of physical propagators and let $\boldsymbol{b} = (1, \ldots, 1, 0, \ldots, 0)$ have P entries equal to one. Let $\boldsymbol{e}_j$ be the unit vector for index j. The first batch contains $\boldsymbol{b} + 2\boldsymbol{e}_j$ for $j = 1, \ldots, \min(P, 3)$. Thus one denominator has power three in each target. For the second batch, define the lower-sector corner

$$\boldsymbol{b}^{\mathrm{low}} = \begin{cases} \boldsymbol{b} - \boldsymbol{e}_1, & P < 5, \\ \boldsymbol{b} - \boldsymbol{e}_1 - \boldsymbol{e}_3, & P \geq 5. \end{cases} \tag{68}$$

Include this corner and up to three single-dot targets, $\boldsymbol{b}^{\mathrm{low}} + \boldsymbol{e}_j$ with $b_j^{\mathrm{low}} = 1$, taking the first available indices in increasing order. The third batch uses the original corner $\boldsymbol{b}$ and contains $\boldsymbol{b} - 2\boldsymbol{e}_j$ for the first three auxiliary entries, or all auxiliary entries when fewer than three exist. When a family has no auxiliary entry, this construction repeats the first batch and we remove the duplicate. These formulas specify the requested integral indices. Their master coefficients are obtained by exact reduction and are not used to construct the model inputs.

J.1 The allowed changes to Kira

For target batch $\mathcal{T}$ and a target $\boldsymbol{a}$, let $S(\boldsymbol{a}) = \{i : a_i > 0\}$ be its positive-index set. One strategy supplies only maximal target sectors:

$$\mathcal{S}_{\max} = \{S(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T},\ S(\boldsymbol{a}) \not\subseteq S(\boldsymbol{a}') \text{ for every } \boldsymbol{a}' \in \mathcal{T}\}. \tag{69}$$

Kira then includes their subsectors. To see why this is a useful candidate, consider the shifted integrals produced by the IBP derivative. A raised index $a_i + 1$ carries a factor a_i, and the other shifts lower an index. If $a_i = 0$, the raising term vanishes; if $a_i < 0$, raising it once leaves it nonpositive. Thus an IBP equation seeded in one sector stays in that sector or its subsectors. The sector property makes this restriction a useful candidate, but it does not establish that the restricted pool contains every relation needed for the prescribed basis. We therefore retain the basis check and the option to repeat the calculation with the full sector set.

Another strategy uses Kira's numerator truncation [5]. In a sector with t positive indices, Kira restricts the inherited numerator bound by

$$s \le \max(1, t - \tau + 1, s_{\text{explicit}}), \tag{70}$$

where τ is the chosen integer cutoff and s_{explicit} is a bound explicitly supplied for that sector. We test $\tau = P - s_c$ and $\tau = P - s_c + 1$, where s_c is the largest target numerator rank. The top-level sector retains its explicit target bound. The first cutoff combined with maximal target sectors forms the deterministic control. The development probes also test $\tau = P - s_c - 1$.

The remaining choices use the original bounds B_c, the smaller bounds $B_c - (1, 0, 1)$ with negative entries replaced by zero, the larger bounds $B_c + (1, 1, 1)$, Kira integral ordering five [13], or the preceding trained equation filter. Together these choices give nine strategies. Before each attempt, every strategy enforces a nonempty seed region in its declared top-level sectors. If $\mathcal{S}_0$ is that sector list and r_{req} is the requested positive-index bound, the actual bound is

$$r_{\text{run}} = \max\left(r_{\text{req}}, \max_{S \in \mathcal{S}_0} |S|\right). \tag{71}$$

Each positive index is at least one, so a seed in sector S has $\sum_{i \in S} a_i \ge |S|$. This proves the lower bound used in Eq. (71). Enforcing this bound prevents an invalid configuration from causing a failed startup before any meaningful reduction can occur. Identical input configurations share one training or validation measurement. If a proposed reduction fails, it retries with native choices at B_c and then at $B_c + (1, 1, 1)$. The recorded cost includes every attempt.

J.2 Fitting and selecting the models

Each graph has its own predictors of $y_{cj} = \log(t_{cj}/t_{c0})$, where t_{cj} is the median complete cost of strategy j on batch c and zero denotes native choices. The nine input features are the target count; maximum dot count and numerator rank; minimum and maximum positive-index counts; mean dot count and numerator rank; number of maximal target sectors; and number of physical propagators. The features are calculated from the requested indices and the family definition, so no reduced coefficients enter the prediction.

We fit ten candidates per graph: one constant predictor, trees of maximum depth one, two and three, a bagged ensemble of sixteen depth-three trees, ridge regressions with penalties $0.1, 1, 10$, and nearest-neighbour predictors with one and three neighbours. Trees use squared-error splits with at least one row in each child. Each ensemble tree receives a bootstrap sample of the training batches and uses all features at each split. This follows bootstrap aggregation [16]. For fitted trees $f_1, \ldots, f_{16}$, the ensemble prediction is

$$\widehat{y}_j(\boldsymbol{f}) = \frac{1}{16} \sum_{b=1}^{16} f_{bj}(\boldsymbol{f}). \tag{72}$$

This average also follows by minimizing $\sum_b (f_{bj}(\boldsymbol{f}) - v_j)^2$ with respect to v_j, using the leaf-mean derivative in Appendix I. That appendix derives the ridge fit and the local-mean estimates used here. Neighbour distances use the training standard deviations; a constant feature uses divisor one. We use two timing repetitions for training and three for validation. Within each batch comparison, every strategy uses the same computer. Different training batches can use different computers, and the training response normalizes each cost to its own batch baseline.

Validation considers decision margins $0, 0.05, 0.15$ in Eq. (63). It chooses the candidate and margin with the lowest complete cost separately for each graph. The controls are native choices, the deterministic sector-and-truncation rule, and the single strategy with the lowest geometric mean training cost for that graph. A fixed strategy can include the older equation filter. We enable the new model only when its validation cost ratio to the fastest control is at most 0.95. Otherwise, the method uses that control. The threshold determines when the model is enabled; statistical interpretation comes from the separately reported test measurements. Unknown graph structures and features outside the fitted range use native choices.

The reserved comparison uses five repetitions of native choices, the deterministic rule, the fixed strategy per graph, and the selected method. The order rotates across batches and repetitions, giving every method the same number of appearances in each run position over the full test. Each run uses the same targets, prescribed basis, executable, and one processor. All methods retain the preceding exact implementation improvements, so the native control uses that same implementation with native selection rules. Full time includes input preparation, prediction, native IBP generation, exact reduction, coefficient checks, cleanup, and retries. Offline fitting, reference discovery, and saving measurement records stay outside the timer. Separate application checks receive the basis without target coefficients and compare their answers only after completion. The calling program and fitted parameters are loaded before the timing trials, and each trial starts a fresh Kira process. The additional application checks also record time with a newly started calling program; they do not supply a matched speed comparison for that condition.

We combine five-run median ratios within each graph and then give the seven graphs equal weight, as derived in Appendix I. The aggregate intervals resample graph groups. Per-graph plot bars show the range across the two reserved kinematic profiles. These ranges show the variation observed in two profiles. The sample is too small to support a general significance claim for the integral type.

K Selective seed expansion and exact basis completion

The seed-expansion study asks whether a smaller first calculation and an exact conversion of its remaining integrals can be cheaper than one large reduction. It uses the seven families in Appendix I and the target definitions in Appendix J. Development uses profile $h = 71$, training uses $h = 72, 73$, validation uses $h = 74$, and the reserved test uses $h = 75, 76$. These are separate rational parameter profiles. The graph structures and target index patterns occur in all splits, and d remains symbolic.

K.1 Changing individual seed bounds

The bounds $B_c = (r_c, s_c, q_c)$ retain the definitions in Appendix I: r_c bounds the sum of positive indices, s_c bounds the numerator rank, and q_c bounds the number of extra denominator powers. Each component starts from the largest value among the requested targets. We first enforce the sector lower bound in Eq. (71). The six new direct reductions use

$$B_c + \boldsymbol{\delta}, \qquad \boldsymbol{\delta} \in \{(1,0,0), (0,1,0), (0,0,1), (1,0,1), (1,1,0), (0,1,1)\}. \tag{73}$$

For a seed with t positive indices, summing $a_i = 1 + (a_i - 1)$ over those indices gives $r = t + q$. The positive-index and extra-power limits can therefore overlap, and different requested triples

can give the same seed region after all limits are applied. The labels therefore identify requested limits rather than necessarily distinct equation sets. Their purpose is to allow selective expansion, such as requesting more numerator identities without also increasing both denominator limits. Sufficiency remains a property to check for the resulting system. Each direct reduction keeps the prescribed-basis check described in Ref. [5]. If it fails, the calculation calls the preceding fixed graph-specific method and includes both attempts in its time.

K.2 Completing a temporary basis

We also test two ways to complete a smaller reduction. The first stage uses B_c and supplies the preferred masters, while allowing Kira to return additional integrals. Kira supports this behaviour when its early master-basis check is disabled [5]. Let M_m denote prescribed masters and let U_u denote additional integrals that remain in the first-stage output. For target I_t, this stage returns exact relations

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} U_u. \tag{74}$$

Here A_{tm} is the coefficient of prescribed master M_m, and C_{tu} is the coefficient of unresolved integral U_u. These coefficients are rational functions of the kinematics and d. The integrals U_u are unresolved in the smaller first-stage system and need not be independent masters of the full family. Their index vectors define the targets of the second reduction, allowing that calculation to focus on the integrals that actually remain. That request uses its own componentwise target bounds plus $(1, 1, 1)$ and again supplies the prescribed basis. One candidate uses the maximal sectors of the U_u and their subsectors; the other keeps the original top-level sectors. Both retain the positive-index lower bound.

If the second reduction succeeds, it gives

$$U_u = \sum_m H_{um} M_m. \tag{75}$$

The coefficient H_{um} multiplies master M_m in the reduction of U_u. Substitution of Eq. (75) into Eq. (74) yields

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} \sum_m H_{um} M_m \tag{76}$$

$$= \sum_m \left(A_{tm} + \sum_u C_{tu} H_{um} \right) M_m. \tag{77}$$

The final coefficient matrix is therefore $A + CH$. Computing its entries exactly in the rational-function field, cancelling common factors, and checking the final integral labels gives a reduction in the prescribed basis. When either reduction fails, the preceding method supplies a full fallback. If no additional integral occurs, the second reduction is unnecessary. All basis conversion and fallback costs enter the timer. Independent reference coefficients check the final result only; the policy and the two reduction stages do not receive them.

K.3 Training, model selection, and comparison

The development set compares the preceding method, six direct expansions, and two basis-completion choices. We retain the preceding method and each case's fastest candidate when it lowers that case's complete time by at least ten percent. This rule selects candidates before training and does not inspect reserved data.

We then fit the ten model types and nine target features defined in Appendix J, separately for each graph. The new response is $\log(t_{cj}/t_{c\mathrm{prev}})$, so it directly measures improvement over the preceding method. Here t_{cj} is the median complete time of candidate j on batch c, and $t_{c\mathrm{prev}}$ is

the corresponding time of the preceding method. Two repetitions supply each training label, and three repetitions supply each validation cost. A fixed strategy minimizes geometric mean training time for each graph. Validation first selects the faster control between that fixed strategy and the preceding method. It then tests the fitted predictors at margins $0, 0.05, 0.15$ and enables one only when it lowers the control's validation time by at least five percent. The fitting formulas and their derivations remain those in Appendices I and J.

The fixed control may call the learned method retained from the preceding study. Its purpose is to isolate the benefit of the new choice among candidates while preserving the earlier improvements in every method.

The chosen graph policy and candidate set are fixed before the reserved tests. Inputs with another graph structure or target features outside the training range use the preceding method. That method retains its own native fallback. Each reserved batch compares native choices, the preceding method, the new fixed strategy, and the selected new method on the same computer. Five repetitions use a rotating order. All methods use the same executable and exact final coefficient checks. The time includes preparation, prediction, Kira, any basis conversion, checking, cleanup, and every retry. Offline training, independent reference generation, and writing benchmark records remain outside these times. The selection program and fitted parameters are loaded before these trials; each attempt starts a fresh Kira process. Separate application tests start a new caller and supply only the targets, family, and prescribed basis. They check the coefficients after the reduction.

We first take the median time for each batch and method, then combine ratios within each graph and give all seven graphs equal weight. The geometric-mean derivation in Appendix I also applies here. Descriptive aggregate intervals resample the seven graph groups. The per-graph bars show the range over the two reserved parameter profiles. These ranges describe the tested profiles and do not establish a gain for all integrals with the same graph.

L Learned choices of integral and propagator order

This appendix specifies the ordering choices and explains why consistent relabelling leaves the integrals unchanged. The comparison uses the family definitions and target patterns in Appendix J. We use parameter profile 81 for development, profiles 82 and 83 for training, profile 84 for validation, and profiles 85 and 86 for the reserved comparison. Each profile uses the rational parameters defined there, while the dimension d remains symbolic. These are new parameter values of known graphs and target patterns.

L.1 What the choice changes

Kira provides eight integral orderings [13]. Order one treats numerator powers as simpler than extra denominator powers, called dots; order two reverses that preference. Orders three and four first compare their sum and then favour numerator powers or dots, respectively. Orders one through four compare sectors by sector number. Orders five through eight use the corresponding index rules but first compare the number of positive indices and favour subsectors of the declared top sectors. We also test the four automatic propagator permutation schemes of Kira [5]. The final coefficients use the original index convention in every comparison.

The development candidates retain the preceding method, test each integral order from two through eight with propagator scheme one, and test propagator schemes two through four with integral order one. These are eleven initial choices. For each development batch, we also combine its fastest nondefault integral order with its fastest nondefault propagator scheme and time that pair. This adds one development calculation per batch. The preceding seed policy remains active in each new choice. It can use truncation, selected seed bounds, or temporary-basis completion

as defined in the preceding appendices. A changed ordering is applied to each Kira attempt in that method. If it fails to reduce to the prescribed basis, we repeat the preceding method with its original settings. Both attempts enter the recorded time. This finite comparison does not search every joint ordering or every possible elimination sequence.

To see why changing labels preserves an integral, let π be a permutation of the N propagator positions. Set $D'_i = D_{\pi(i)}$ and $a'_i = a_{\pi(i)}$. Reindexing the product gives

$$\prod_{i=1}^{N} (D'_i)^{-a'_i} = \prod_{i=1}^{N} D_{\pi(i)}^{-a_{\pi(i)}} = \prod_{j=1}^{N} D_j^{-a_j}, \qquad I_{D'}(\mathbf{a}') = I_D(\mathbf{a}). \tag{78}$$

Here I_D denotes the integral defined by propagators D_i. Since the relabelling also leaves the measure unchanged, it preserves the integral. Changing the priority of integrals instead changes the sequence in which Kira solves the IBP equations. If $A\mathbf{I} = 0$ denotes those equations and P is the permutation matrix that reorders the integral columns, then

$$\mathbf{J} = P\mathbf{I}, \qquad AP^{-1}\mathbf{J} = A\mathbf{I} = 0. \tag{79}$$

The permutation therefore represents the same equations with a different column order. In a finite calculation, however, the ordering can also affect which equations are selected, so the equivalence of representations does not replace the checks of completeness and exact output coefficients. We require every target to reduce to the prescribed master basis and compare every output coefficient with an independent expanded-seed reduction. The predictor receives neither those coefficients nor the reference reduction steps.

L.2 Training, validation, and timing

For each development batch, we retain the fastest candidate if its full time is at least ten percent below the preceding method. We also retain the preceding method. The resulting candidate set is fixed before training. We fit the same ten predictor types per graph as in Appendix K, using the same nine features and complete-time log-ratio labels. Training has two measurements per candidate and batch; validation has three. Validation chooses a fixed control and enables a predictor only when it saves at least five percent against that control. We freeze the models, controls, and selection requirements before using the reserved profiles.

Each application selects its ordering before Kira creates integral weights and generates equations. The prediction is made once per requested batch. The selected method retains the earlier seed policy, and Kira performs all algebraic operations. The decision selects a setting for the complete reduction before it starts. Kira continues to choose individual row operations, as required by the earlier row-model comparison.

The reserved comparison measures native choices, the preceding method, the training-selected fixed setting for each graph, and the new selected method. It uses five repetitions with rotating execution order. The same worker handles all methods for each batch. The timer includes prediction, input preparation, native equation generation, exact algebra, basis conversion where needed, coefficient checks, fallback, and cleanup. The caller and models are already loaded, and each attempt starts a new Kira process. Offline development, reference construction, training, and record writing are excluded. Additional applications receive the target indices and master basis without reference coefficients. Their results are checked independently after the reduction.

M Selecting the number of Kira workers

The worker-count study retains the seed strategy from Section 5.3 and varies only the number of Kira workers used for a requested batch. The families and target patterns are those in Appendix J.

Parameter profile 91 supplies the development test, profiles 92 and 93 supply training, profile 94 supplies validation, and profiles 95 and 96 supply the reserved comparison. The dimension remains symbolic. Target patterns overlap across these sets; the rational kinematic values differ.

M.1 Choices and exactness

The preceding learned method chooses seed bounds and can reduce through a temporary basis. We retain that method and let a new policy choose one, two, or four Kira workers for each requested batch. The choice applies to each Kira attempt, including a temporary-basis conversion. Kira supports parallel symbolic reduction [4]. Changing the worker count changes how operations are scheduled, while each operation continues to use exact rational arithmetic. Every output must reach the prescribed master basis and agree with an independent expanded-seed reduction. If a new choice fails, its time and the repeat with the preceding one-worker method both enter the complete time.

Let $A\mathbf{I} = 0$ denote the IBP system. An exact elimination step left-multiplies it by an invertible elementary matrix E, so

$$A\mathbf{I} = 0 \quad \Longleftrightarrow \quad EA\mathbf{I} = 0. \tag{80}$$

Multiplication by E proves the forward implication, and multiplication by E^{-1} proves the reverse. Repeating valid steps therefore preserves the system, provided parallel execution respects their dependencies. Kira manages those dependencies through its existing workers, while the predictor sets only their number. This argument preserves relations but does not prove that a finite seed set is sufficient. The basis and coefficient checks therefore remain part of every measured reduction.

M.2 Cost labels and selection

Write x for the nine target features defined in Appendix K, and $q \in \{1, 2, 4\}$ for the worker count. For training batch b, let $t_{b,q}$ be the median complete time of two measurements. The training label is

$$y_{b,q} = \log(t_{b,q}/t_{b,1}). \tag{81}$$

Because the logarithm is strictly increasing and $t_{b,1} > 0$ is common to all choices for that batch, minimizing this label also minimizes $t_{b,q}$. We fit the same ten predictor types per graph as in Appendix K. They include constants, trees, bagged trees, regularized linear fits, and nearest-neighbour fits.

The training-selected fixed control minimizes $\sum_b t_{b,q}$. Validation uses three measurements per choice and batch. It first selects the faster of that fixed control and the preceding method, using the sum of batch medians. It then compares each predictor with decision margins 0, 0.05, and 0.15 in log relative cost. A predictor is enabled only if its summed median cost is at least two percent lower than the control and its summed cost is lower in at least two of the three validation repetitions. Otherwise the control stays active. This objective measures the time for the workload by summing batch costs. It therefore gives a different weighting from the geometric mean that treats graphs equally. The selected controls and models remain fixed throughout the reserved tests. Unknown graph structures or features outside the training range use the preceding method.

M.3 Complete-time comparisons

The reserved comparison has 36 batches and five repetitions. It measures native choices with one worker, the preceding learned method with one worker, that same method with four workers throughout, a training-selected fixed worker count per graph, and the final selected method. All methods use the same integrated executable and its common earlier optimizations. The native control uses Kira's native seed and reduction choices in that executable. All methods

run sequentially in the same four-core allocation. Execution order rotates between repetitions. Thus the always-four-worker control measures the cost of using the extra cores without target-dependent selection.

Full time includes model decisions, input preparation, Kira startup, equation generation, exact algebra, any basis conversion, coefficient checks, fallback, and cleanup. The caller and models are already loaded, while every attempt starts a new Kira process. Offline reference construction, training, and record writing are excluded. CPU time sums user and system time of the caller and its completed child processes. This measures the CPU time actually consumed. Every method has four reserved cores, including the methods that use only one worker. The comparison reports both the sum of batch medians and the graph-weighted geometric ratios defined in the earlier appendices. Additional applications receive the targets and basis without reference coefficients and are checked after reduction.

References

[1] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[2] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[3] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[4] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[5] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection. *Computer Physics Communications*, 322:109999, 2026. arXiv:2505.20197.

[6] Zhuo-Yang Song, Tong-Zhi Yang, Qing-Hong Cao, Ming-xing Luo, and Hua Xing Zhu. Explainable AI-assisted optimization for Feynman integral reduction, 2025. arXiv:2502.09544.

[7] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[8] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.06114.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[11] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[12] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman & Hall, 1984. Publisher record.

[13] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[14] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. publisher DOI.

[15] Richard Bellman. On the theory of dynamic programming. *Proceedings of the National Academy of Sciences*, 38(8):716–719, 1952. publisher DOI.

[16] Leo Breiman. Bagging predictors. *Machine Learning*, 24:123–140, 1996. Publisher record.